

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»**

**Факультет прикладної математики
Кафедра системного програмування і
спеціалізованих комп'ютерних систем**

«На правах рукопису»
УДК _____

До захисту допущено:
Завідувач кафедри
_____ Віталій РОМАНКЕВИЧ
«___» _____ 2025 р.

Магістерська дисертація

на здобуття ступеня магістра

**за освітньо-професійною програмою «Системне програмування та
спеціалізовані комп'ютерні системи»**

зі спеціальності 123 «Комп'ютерна інженерія»

**на тему: «Система автоматизації розгортання ШІ-застосунків у
хмарному середовищі»**

Виконав:

студент II курсу, групи KB-42 мп
Оніщук Андрій Олегович _____

Науковий керівник:

с.н.с-к.т.н., доц.,
Тесленко Олександр Кирилович _____

Рецензент:

Засвідчую, що у цій магістерській
дисертації немає запозичень з праць
інших авторів без відповідних
посилань.

Студент _____

Київ – 2025 року

**Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»**

Факультет прикладної математики

Кафедра системного програмування і спеціалізованих комп'ютерних систем

Рівень вищої освіти – другий (магістерський)

Спеціальність – 123 «Комп'ютерна інженерія»»

Освітньо-професійна програма «Системне програмування та спеціалізовані комп'ютерні системи»

ЗАТВЕРДЖУЮ

Завідувач кафедри

_____ Віталій РОМАНКЕВИЧ

«___» _____ 2024 р.

**ЗАВДАННЯ
на магістерську дисертацію студенту
Оніщуку Андрію Олеговичу**

1. Тема дисертації «Система автоматизації розгортання ШІ-застосунків у хмарному середовищі», науковий керівник дисертації Тесленко Олександр Кирилович, с.н.с-к.т.н., доцент кафедри СПіСКС, затверджені наказом по університету від «6» листопада 2025 р. №4836-с
2. Термін подання студентом дисертації 12 грудня 2025
3. Об'єкт дослідження: процеси розгортання, оновлення та експлуатації моделей штучного інтелекту в хмарно-орієнтованих інфраструктурах.
4. Вихідні дані: контейнери Docker і Docker Compose, мова програмування Python, середовища з підтримкою GPU та NVIDIA Container Toolkit, DevOps/MLOps-підходи, хмарні системи.
5. Перелік завдань, які потрібно розробити: проаналізувати сучасні підходи до автоматизації розгортання ШІ-додатків у хмарному середовищі, дослідити архітектурні рішення та інструменти для автоматичного розгортання ШІ-систем, запропонувати систему автоматизації розгортання ШІ-застосунків у хмарному середовищі, провести тестування запропонованої системи та оцінити ефективність запропонованого підходу.

6. Орієнтовний перелік графічного (ілюстративного) матеріалу - презентація

7. Орієнтовний перелік публікацій

7.1. Прикладна математика та комп'ютинг «XVIII науково-практична конференція магістрантів та аспірантів ПМК-2025 факультету прикладної математики».

7.2. XI Міжнародна науково-практична конференція «GLOBAL TRENDS IN SCIENCE AND EDUCATION»

9. Дата видачі завдання 1 листопада 2024р.

Календарний план

№ з/п	Назва етапів виконання магістерської дисертації	Термін виконання етапів магістерської дисертації	Примітка
1.	Вивчення літератури за тематикою дисертації	25.09.2024	
2.	Підготовка середовища та початок розробки системи	13.12.2024	
3.	Підготовка змісту та вступу магістерської дисертації	05.03.2025	
4.	Підготовка реферату магістерської дисертації	19.05.2025	
5.	Підготовка матеріалів першого розділу магістерської дисертації	09.07.2025	
6.	Підготовка матеріалів другого розділу магістерської дисертації	11.08.2025	
7.	Підготовка матеріалів третього розділу магістерської дисертації	05.09.2025	
8.	Тестування розробленої системи	25.09.2025	
9.	Підготовка матеріалів четвертого розділу магістерської дисертації	15.10.2025	
10.	Підготовка тез доповіді на конференції	01.11.2025	
11.	Виступи на конференціях	23.11.2025	
12.	Оформлення документації магістерської дисертації	26.11.2025	
13.	Попередній розгляд магістерської дисертації на кафедрі	12.12.2025	

Студент

Андрій ОНІЩУК

Науковий керівник

Олександр ТЕСЛЕНКО

РЕФЕРАТ

Актуальність теми. Сучасний розвиток цифрових технологій та стрімке зростання обсягів даних зумовлюють активне впровадження систем штучного інтелекту в різні сфери діяльності — від бізнесу й промисловості до освіти та державного управління. Ефективне функціонування таких систем потребує надійних рішень для їх розгортання, оновлення та масштабування у хмарному середовищі. Традиційні підходи до розгортання моделей машинного навчання часто виявляються трудомісткими та малоефективними при частих змінах моделей або великій кількості сервісів. Це створює потребу у використанні сучасних DevOps- та MLOps-практик, які забезпечують безперервну інтеграцію, доставку та автоматичне оновлення ШІ-рішень. Тому розробка підходу та демонстраційної архітектури для автоматизованого розгортання та оновлення AI-додатків у хмарному середовищі є актуальним завданням, що сприяє підвищенню надійності, гнучкості та ефективності впровадження інтелектуальних систем як у наукових, так і у промислових застосуваннях.

Об'єктом дослідження є процес автоматизованого розгортання та супроводу систем штучного інтелекту у хмарному середовищі.

Предметом дослідження є методи, підходи та програмні інструменти автоматизації розгортання систем штучного інтелекту у хмарному середовищі, зокрема із використанням технологій контейнеризації, оркестрації, а також принципів DevOps і MLOps.

Мета роботи: аналіз сучасних підходів до автоматизації розгортання систем штучного інтелекту у хмарному середовищі, дослідженні ефективних інструментів контейнеризації, оркестрації та CI/CD, а також у розробці та впровадженні програмного рішення, що забезпечує автоматизоване розгортання, оновлення й масштабування моделей машинного навчання в хмарній інфраструктурі.

Наукова новизна полягає в наступному:

1. Запропоновано структуру контейнеризованої архітектури, що забезпечує гнучке оновлення та масштабування моделей машинного навчання.
2. Розроблено підхід до автоматизованого розгортання систем штучного інтелекту у хмарному середовищі з використанням контейнеризації та CI/CD-конвеєрів.
3. Реалізовано прототип програмного комплексу, який поєднує DevOps і MLOps-практики для безперервного розгортання та моніторингу ШІ-додатків.

Практична цінність отриманих у роботі результатів полягає в розробці ефективного підходу до автоматизації розгортання ШІ-додатків у хмарному середовищі. Запропоноване рішення дозволяє спростити процес оновлення, масштабування та моніторингу моделей машинного навчання, що знижує витрати часу та ресурсів при впровадженні інтелектуальних систем. Реалізований прототип може бути використаний як основа для навчальних, дослідницьких або комерційних проектів, де потрібна швидка інтеграція моделей штучного інтелекту у хмарну інфраструктуру.

Апробація роботи. Основні положення і результати роботи були представлені та обговорені на XVIII науково-практичній конференції магістрантів та аспірантів «Прикладна математика та комп'ютинг» ПМК-2025 (Київ, 19-21 листопада 2025 р.) та на XI Міжнародній науково-практичній конференції «GLOBAL TRENDS IN SCIENCE AND EDUCATION» (Київ, 17-19 листопада 2025 р.)

Структура та обсяг роботи. Магістерська дисертація складається зі вступу, чотирьох розділів та висновків.

У *вступі* наведено загальну характеристику дослідження, обґрунтовано актуальність теми, визначено об'єкт, предмет, мету та основні завдання роботи, розкрито наукову новизну, практичну цінність та апробацію отриманих результатів.

У першому розділі здійснено аналіз сучасних підходів до автоматизації розгортання систем штучного інтелекту у хмарному середовищі. Розглянуто роль хмарних технологій у розвитку ШІ-рішень, проведено огляд існуючих методів автоматизації та виявлено основні проблеми й обмеження сучасних підходів.

У другому розділі досліджено архітектурні рішення, технології та інструменти, що застосовуються для автоматичного розгортання ШІ-систем. Розглянуто використання контейнеризації, оркестрації, DevOps- і MLOps-практик, а також підходи до безперервного оновлення та масштабування моделей у хмарному середовищі.

У третьому розділі описано реалізоване програмне забезпечення: описано структуру системи та її компоненти, розкрито підхід до реалізації контейнеризованих модулів і CI/CD-конвеєра, а також представлено користувацький інтерфейс розробленої системи.

У четвертому розділі подано результати експериментального тестування створеної системи. Виконано аналіз ефективності, продуктивності та масштабованості рішення, а також здійснено порівняння отриманих результатів із традиційними підходами до розгортання ШІ-додатків.

У висновках підсумовано результати дослідження, сформульовано основні висновки та наведено рекомендації щодо подальшого розвитку і вдосконалення розробленої системи.

Робота представлена на * аркушах, містить посилання на список використаних літературних джерел.

Ключові слова: штучний інтелект, хмарні технології, автоматизація розгортання, контейнеризація, оркестрація, нейронні мережі, модель машинного навчання, хмарне середовище.

ABSTRACT

Relevance of the topic. The rapid development of digital technologies and the exponential growth of data volumes have led to the widespread adoption of artificial intelligence systems across various domains — from business and industry to education and public administration. The effective operation of such systems requires reliable solutions for deployment, updating, and scaling within cloud environments. Traditional approaches to deploying machine learning models often prove labor-intensive and inefficient, especially under conditions of frequent model updates or many interdependent services. This creates a demand for modern DevOps and MLOps practices, which enable continuous integration, delivery, and automated updating of AI solutions. Therefore, the development of an approach and a demonstrational architecture for automated deployment and updating of AI applications in a cloud infrastructure is a relevant task that contributes to improving the reliability, flexibility, and efficiency of intelligent systems in both scientific and industrial applications.

The object of research is the process of automated deployment and maintenance of artificial intelligence systems in a cloud environment.

The subject of research is the methods, approaches, and software tools for automating the deployment of artificial intelligence systems in a cloud environment, particularly using containerization technologies, orchestration mechanisms, as well as DevOps and MLOps principles.

The purpose of the work is to analyze contemporary approaches to automating the deployment of artificial intelligence systems in cloud environments, to investigate effective containerization, orchestration, and CI/CD tools, and to develop and implement a software solution that enables automated deployment, updating, and scaling of machine learning models in cloud infrastructure.

Scientific novelty of the work consists in the following:

1. Containerized architecture has been proposed that ensures flexible updating and scaling of machine learning models.
2. An approach to automated deployment of artificial intelligence systems in cloud environments based on containerization and CI/CD pipelines has been developed.
3. A prototype of a software system integrating DevOps and MLOps practices for continuous deployment and monitoring of AI applications has been implemented.

The practical value of the results lies in the development of an effective approach to automating the deployment of AI applications in cloud environments. The proposed solution simplifies the processes of updating, scaling, and monitoring machine learning models, reducing time and resource costs associated with integrating intelligent systems. The implemented prototype can serve as a foundation for educational, research, or commercial projects requiring rapid integration of artificial intelligence models into cloud infrastructures.

Approbation of the work. The main results and findings were presented and discussed at the XVIII Scientific and Practical Conference of Master's and Postgraduate Students "Applied Mathematics and Computing" PMK-2025 (Kyiv, November 19–21, 2025) and at the XI International Scientific and Practical Conference "Global Trends in Science and Education" (Kyiv, November 17–19, 2025).

Structure and scope of the work. The master's thesis consists of an introduction, four chapters, and conclusions.

The introduction provides a general overview of the research, substantiates the relevance of the topic, defines the object, subject, purpose, and tasks of the work, and outlines the scientific novelty, practical significance, and approbation of the obtained results.

The first chapter analyzes contemporary approaches to automating the deployment of artificial intelligence systems in cloud environments, explores the role of cloud technologies, and identifies current limitations and challenges.

The second chapter examines architectural solutions, technologies, and tools used for automated deployment of AI systems, including containerization, orchestration, DevOps and MLOps practices, as well as approaches to continuous updating and scaling of models.

The third chapter describes the implemented software system, its structure and components, the design of containerized modules and the CI/CD pipeline, as well as the developed user interface.

The fourth chapter presents the results of experimental testing of the system, analyzes its efficiency, performance, and scalability, and compares the obtained results with traditional deployment approaches.

The conclusions summarize the outcomes of the research, outline the main findings, and provide recommendations for further development and improvement of the system.

The thesis is presented on ____ pages and includes references to the list of used literature sources.

Keywords: artificial intelligence, cloud technologies, deployment automation, containerization, orchestration, neural networks, machine learning model, cloud environment.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ	12
1. АНАЛІЗ СУЧАСНИХ ПІДХОДІВ ДО АВТОМАТИЗАЦІЇ РОЗГОРТАННЯ ШІ-ДОДАТКІВ У ХМАРНОМУ СЕРЕДОВИЩІ	13
1.1 Поняття хмарних технологій та їх роль у розгортанні штучного інтелекту. 13	
1.2 Огляд сучасних методів автоматизації та розгортання ШІ-додатків у хмарній інфраструктурі	17
1.3 Аналіз проблем сучасних методів розгортання та підтримки моделей у хмарному середовищі	21
Висновки до розділу.....	26
2 ДОСЛІДЖЕННЯ АРХІТЕКТУРНИХ РІШЕНЬ ТА ВИБІР ІНСТРУМЕНТІВ ДЛЯ АВТОМАТИЧНОГО РОЗГОРТАННЯ ШІ-СИСТЕМИ.....	29
2.1 Огляд інструментів контейнеризації та оркестрації	29
2.2 Тренування нейромережових моделей у контейнеризованому середовищі..	42
2.3 Особливості впровадження DevOps і MLOps-підходів для AI-додатків	49
2.4 Способи виконання безперервного оновлення та масштабування моделей у хмарному середовищі	53
Висновки до розділу.....	68
3 ОПИС РЕАЛІЗОВАНОЇ ХМАРНО-ОРІЄНТОВАНОЇ СИСТЕМИ ТА ЇЇ КОМПОНЕНТІВ	71
3.1 Характеристика системи та основних компонентів	71
3.2 Підхід до реалізації контейнеризованих модулів	74
3.3 Реалізація автоматичного тренування нейронних моделей	79
3.4 Налаштування CI/CD-конвеєра для автоматичного оновлення моделей.....	85
3.5 Реалізовані способи моніторингу розгортання та тренування.....	87
Висновки до розділу.....	98
4 ВИПРОБУВАННЯ РОЗРОБЛЕНОЇ СИСТЕМИ ТА АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ	100
4.1 Тестування роботи розробленої системи.....	100
4.2 Оцінка продуктивності, надійності та масштабованості системи.....	108

4.3 Порівняння отриманих результатів зі звичними підходами розгортання ...	111
Висновки до розділу.....	112
ВИСНОВКИ	114
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ	116

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

AI (Artificial Intelligence) — штучний інтелект.

API (Application Programming Interface) — прикладний програмний інтерфейс.

CI/CD (Continuous Integration / Continuous Delivery) — безперервна інтеграція та безперервна доставка програмного забезпечення.

CPU (Central Processing Unit) — центральний процесор.

CUDA (Compute Unified Device Architecture) — платформа паралельних обчислень NVIDIA.

DevOps (Development and Operations) — методологія інтеграції процесів розробки та експлуатації програмного забезпечення.

DNS (Domain Name System) — система доменних імен.

Dockerfile — конфігураційний файл для побудови контейнера Docker.

GPU (Graphics Processing Unit) — графічний процесор.

HTTP (HyperText Transfer Protocol) — протокол передачі гіпертексту.

JSON (JavaScript Object Notation) — текстовий формат подання структурованих даних.

MLOps (Machine Learning Operations) — підхід до автоматизації життєвого циклу моделей машинного навчання.

RAM (Random Access Memory) — оперативна пам'ять.

REST (Representational State Transfer) — архітектурний стиль побудови веб-сервісів.

UML (Unified Modeling Language) — універсальна мова моделювання.

ПЗ — програмне забезпечення.

ІІІ — штучний інтелект.

1. АНАЛІЗ СУЧАСНИХ ПІДХОДІВ ДО АВТОМАТИЗАЦІЇ РОЗГОРТАННЯ ІІІ-ДОДАТКІВ У ХМАРНОМУ СЕРЕДОВИЩІ

1.1 Поняття хмарних технологій та їх роль у розгортанні штучного інтелекту

Хмарні технології — це сукупність моделей, сервісів і практик, що забезпечують доступ до обчислювальних ресурсів (процесорної потужності, пам'яті, дискового простору, мережі, платформних сервісів) за потреби через мережу. Головна риса хмарних підходів — абстрагування апаратного забезпечення від користувача шляхом надання ресурсів у вигляді керованих сервісів із динамічним розподілом та білінгом за споживанням. Такий підхід дозволяє користувачам і розробникам фокусуватися на логіці додатків і алгоритмах замість управління інфраструктурою.

Класифікація хмарних сервісів за моделлю надання (служби) є важливою для розуміння їх ролі в екосистемі розробки та розгортання ІІІ. Існують три основні моделі: інфраструктура як сервіс (IaaS), платформа як сервіс (PaaS) та програмне забезпечення як сервіс (SaaS). IaaS надає базові обчислювальні ресурси — віртуальні машини, блокове зберігання, мережеві компоненти — що дає максимальну гнучкість при побудові інфраструктури під потреби конкретних робочих навантажень. PaaS забезпечує середовище виконання й служби (наприклад, керовані бази даних, сервіси для розгортання додатків), що спрощує розробку й автоматизацію. SaaS пропонує готові застосунки через інтернет. Для задач машинного навчання і розгортання моделей ключову роль відіграють IaaS і PaaS, оскільки вони дозволяють керувати як апаратними ресурсами (CPU/GPU/TPU), так і службами для зберігання, обробки даних, CI/CD і моніторингу.

Друга важлива класифікація — моделі розгортання: публічні хмари, приватні хмари, гібридні й мультихмарні архітектури. Публічні хмари пропонують швидку масштабованість і широкий набір керованих сервісів, тоді як приватні хмари дають змогу зберігати контроль над апаратурою, політиками безпеки та локальним зберіганням даних. Гібридні та мультихмарні підходи поєднують переваги обох світів і дозволяють розподілити робочі навантаження

між різними типами інфраструктур у відповідності зі специфічними вимогами (наприклад, чутливі дані — у приватній інфраструктурі; масштабні тренування — у публічній хмарі з потужними GPU). Для масштабних ML-проектів мультихмарність також забезпечує відмовостійкість і можливості оптимізації витрат.

Роль хмарних технологій у розгортанні штучного інтелекту є багатогранною. По-перше, хмари надають доступ до спеціалізованого апаратного забезпечення (GPU, TPU, прискорювачі для інференсу), яке є необхідним для ефективного тренування великих нейронних мереж і швидкого інференсу в продуктивному середовищі. По-друге, хмарні платформи спрощують організацію зберігання великих обсягів даних, надаючи масштабовані об'єкти сховищ, розподілені файлові системи й бази даних, що важливо для підготовки й обробки навчальних наборів. Третій аспект — це наявність інтегрованих сервісів для оркестрації робочих навантажень (контейнерні платформи, керовані кластери Kubernetes), CI/CD та автоматизації, що безпосередньо полегшує побудову відтворюваних і керованих MLOps-процесів.

З погляду життєвого циклу моделі, хмари впливають на ключові етапи: збір і зберігання даних, препроцесінг, навчання, валідація, розгортання й моніторинг. Хмара дозволяє організувати канали передачі даних та пайплайни обробки, де обчислювальні завдання автоматично масштабуються згідно з навантаженням. У контексті CI/CD для моделей машинного навчання це означає можливість автоматичного запуску тренувань у відповідь на оновлення наборів даних або коду, автоматичну валідацію якості моделей і подальше розгортання перевірених артефактів у продакшн-середовище без тривалих простоїв сервісів.

Переваги застосування хмарних технологій для AI-проектів можна узагальнити так: еластичність ресурсів, швидке розгортання середовищ для розробки та тестування, стандартизація середовища через контейнери, централізоване зберігання артефактів і даних, а також доступ до керованих сервісів (автоматичне масштабування, керовані БД, логінг, моніторинг). Ці

можливості знижують бар'єри для експериментів, пришвидшують інновації й дозволяють організаціям реагувати на зміни моделей і даних із мінімальним ручним втручанням.

Водночас хмарні підходи мають специфічні виклики щодо застосування в AI. До ключових проблем належать питання захисту даних і конфіденційності, латентність при обробці даних у реальному часі, управління витратами при тривалих тренуваннях на прискорювачах, а також складності в забезпеченні повторюваності експериментів у гетерогенних середовищах [8]. Правильна архітектура повинна враховувати ці аспекти: наприклад, реалізовувати шифрування на рівні зберігання та передачі даних, розподіляти важкі обчислення у сегментах хмари з оптимальним співвідношенням ціни і продуктивності, і застосовувати інструменти для версіонування даних і моделей (data versioning, model registry).

Практика розгортання AI-додатків у хмарі значною мірою базується на поєднанні двох ключових технологічних парадигм: контейнеризації та оркестрації. Контейнери забезпечують відтворюваність середовища виконання: залежності, бібліотеки й конфігурації пакуються разом із сервісом, що мінімізує проблеми, що програми працюють на одному комп'ютері, але не працюють при змінні пристрою або середовища. Оркестратори додають можливості управління життєвим циклом контейнерів: деплой, масштабування, автоматичне відновлення, балансування навантаження. Таке поєднання створює основу для безперервного розгортання і оновлення моделей, коли нові версії артефактів плавно вводяться в роботу без інтERRUPTІЇ сервісів.

Інший важливий елемент — автоматизація CI/CD для моделей (MLOps-процеси). Уміння інтегрувати інструменти для автоматичної побудови, тестування та деплоя моделей дозволяє значно скоротити час від створення нового алгоритму до його доступності для кінцевих користувачів. CI/CD-пайплайни у хмарному середовищі можуть включати кроки для автоматичного запуску тренувань, валідації метрик якості моделі, збереження артефактів у

репозиторії моделей та автоматичного оновлення сервісів інференсу згідно політик (наприклад, blue/green або canary deploy) [2].

У контексті розгортання служби інференсу хмари пропонують декілька архітектурних патернів, кожен із яких має власні переваги і обмеження. Патерн "batch inference" підходить для сценаріїв пакетної обробки великих масивів даних; "online inference" (сервісний підхід) забезпечує низьку затримку у відповідь на запити користувачів; "streaming inference" інтегрується з потоковими системами для негайної обробки даних; "serverless inference" дозволяє платити лише за фактичний час виконання, але може мати проблеми з холодним стартом і обмеженнями по ресурсах. Вибір патерну залежить від вимог до латентності, витрат і надійності. Хмарні платформи часто надають інструменти для впровадження всіх цих патернів, що робить їх універсальним середовищем для AI-додатків різної природи.

Окрему увагу слід приділити управлінню життєвим циклом даних у хмарі. Якісне розгортання AI-рішень передбачає не лише управління моделями, але й наявність механізмів для версонування, відстеження походження та чистоти даних, їхньої анонімізації та доступності для відтворюваних експериментів. Хмарні сервіси для збереження й обробки даних (об'єктні сховища, потокові інструменти, керовані бази даних) взаємодіють із CI/CD-пайплайнами і системами моніторингу, створюючи єдиний механізм для контролю якості даних і моделей.

На завершення варто відзначити, що хмарні технології не лише полегшують технічну реалізацію AI-систем, але й формують нові операційні підходи до їхнього життєвого циклу. Вони дозволяють встановити інфраструктурні стандарти, що сприяють відтворюваності експериментів, безперервному вдосконаленню моделей і швидкій інтеграції інновацій у виробництво. Однак для досягнення цих переваг необхідно відповідально підходити до проєктування архітектури, політик безпеки, управління витратами та контролю якості даних — тільки в цьому випадку хмара стане не просто

середовищем розміщення, а реальним механізмом підвищення ефективності AI-рішень.

1.2 Огляд сучасних методів автоматизації та розгортання ШІ-додатків у хмарній інфраструктурі

Розгортання штучного інтелекту (ШІ) у хмарному середовищі передбачає використання комплексу технологій, інструментів і методів, які забезпечують повний життєвий цикл моделі — від розробки та навчання до оновлення, масштабування та моніторингу. Сучасні підходи до автоматизації орієнтовані на зменшення участі людини у процесі розгортання, мінімізацію часу виходу нових версій моделей у продуктивне середовище, а також на забезпечення стабільності, передбачуваності та відтворюваності процесів.

Автоматизація процесів розгортання ШІ-додатків у хмарному середовищі є однією з ключових тенденцій сучасної інформаційної інженерії. Зростання складності моделей, збільшення обсягів даних та вимоги до швидкого оновлення систем зумовили необхідність створення гнучких підходів, які дозволяють мінімізувати людське втручання у життєвий цикл штучного інтелекту.

Сучасні методи автоматизації охоплюють як архітектурні принципи побудови систем, так і алгоритмічні стратегії управління процесами розгортання, масштабування, моніторингу й оновлення моделей машинного навчання. Їхня мета полягає у створенні наскрізного процесу, в якому кожен етап — від збору даних до експлуатації моделі — може бути відтворений, контрольований і модифікований без суттєвих витрат часу та ресурсів.

Першим кроком до автоматизації ШІ є впровадження концепції CI/CD (Continuous Integration / Continuous Delivery), яка забезпечує постійне оновлення програмних компонентів і моделей без переривання роботи системи.

У контексті штучного інтелекту цей підхід трансформується у CICD for ML, де об'єктами автоматизації стають не лише програмні модулі, а й:

- скрипти підготовки даних;

- конвеєри навчання та валідації;
- моделі машинного навчання;
- метадані експериментів.

Алгоритми CI/CD у MLOps ґрунтуються на використанні тригерів подій (наприклад, оновлення даних чи зміна коду моделі), які автоматично запускають відповідні сценарії: повторне навчання, тестування, версіонування та розгортання моделі у виробничому середовищі [2].

Такий підхід мінімізує людські помилки, зменшує час між тренуванням і деплоєм, а також забезпечує контроль якості на кожному етапі життєвого циклу.

MLOps (Machine Learning Operations) є еволюційним розвитком DevOps-практик і спрямований на автоматизацію повного життєвого циклу моделі — від отримання даних до моніторингу результатів у реальному часі [3].

Основні принципи MLOps включають:

- Відтворюваність (Reproducibility) — можливість повторити експерименти та результати моделі за однакових умов;
- Автоматизація (Automation) — зменшення ручних процесів у підготовці, навчанні та розгортанні;
- Спостережуваність (Observability) — моніторинг метрик точності, продуктивності та стабільності;
- Керованість версіями (Versioning) — контроль змін моделей, наборів даних і конфігурацій;
- Безперервне оновлення (Continuous Training) — повторне навчання моделей при зміні вхідних даних або умов середовища.

Математично процес MLOps можна розглядати як багатоконтурну систему керування з адаптивним зворотним зв'язком, де основним параметром регуляції є якість передбачень моделі, а зовнішні збурення — нові дані, оновлені гіперпараметри або зміни архітектури.

На сучасному етапі автоматизація в MLOps включає низку алгоритмічних і аналітичних підходів, які дозволяють оптимізувати процес розгортання та підтримки моделей.

Один з таких підходів має назву AutoML (Automated Machine Learning). AutoML використовує метаалгоритми для автоматичного підбору архітектури моделей, гіперпараметрів і методів попередньої обробки даних. Це дозволяє скоротити людську участь на етапах тренування та валідації.

Для автоматизації таких процесів застосовуються:

- Байєсівська оптимізація — для пошуку оптимальних гіперпараметрів;
- Еволюційні алгоритми — для вибору архітектури нейронних мереж;
- Методи підкріплення (Reinforcement Learning) — для оптимізації гіперпараметрів у динамічних середовищах.

У хмарних середовищах AutoML-системи інтегруються з пайплайнами CI/CD, що дозволяє автоматично тренувати та оновлювати моделі на основі нових даних.

У системах автоматичного розгортання застосовуються алгоритми автоматичного масштабування та самовідновлення (AutoDeploy і Self-Healing Algorithms).

Вони аналізують навантаження на сервіси, кількість запитів, час відгуку та продуктивність моделей. На основі цього застосовується адаптивне масштабування — збільшення або зменшення кількості інстансів сервісу.

Принципово це реалізується через алгоритми динамічного управління ресурсами, подібні до класичних систем зворотного зв'язку, де коефіцієнти регулювання (наприклад, PID-контролери) визначаються емпірично або на основі машинного навчання.

Також розглянемо стратегії автоматизації Canary Deployment і Blue-Green Deployment [1]. Ці стратегії автоматизації використовуються для поступового оновлення моделей без зупинки сервісу.

Canary Deployment передбачає розгортання нової версії моделі для обмеженої кількості користувачів, із подальшим аналізом метрик ефективності. Якщо результати задовільні — оновлення розповсюджується на весь трафік.

Blue-Green Deployment використовує дві ідентичні інфраструктури — “blue” (поточну) та “green” (оновлену). Перемикання між ними здійснюється автоматично після успішного тестування [1].

Для контролю якості розгортаних моделей застосовуються алгоритми A/B-тестування, які порівнюють продуктивність різних версій моделей на однакових даних. Рішення про оновлення моделі приймається автоматично на основі статистично значущої різниці у метриках.

Також у хмарних III-додатках важливим є постійний моніторинг зміни розподілу даних (data drift) та зміни залежностей між ознаками і цільовими змінними (concept drift).

Для виявлення таких змін використовуються:

- методи статистичного контролю (наприклад, Kullback–Leibler divergence);
- класифікаційні ансамблі (які визначають відхилення предиктивних розподілів);
- онлайн-алгоритми навчання, здатні динамічно адаптуватися до нових умов.

Важливим аспектом є вибір архітектури розгортання. У традиційних підходах (Serverful) усі ресурси заздалегідь резервуються та масштабуються вручну. Натомість сучасні Serverless-архітектури базуються на автоматичному виділенні ресурсів «на вимогу».

Функціональне програмування у стилі Function-as-a-Service (FaaS) (наприклад, AWS Lambda, Google Cloud Functions, Azure Functions) дозволяє запускати обчислення лише тоді, коли вони потрібні, що значно підвищує ефективність і гнучкість.

Serverless-підхід особливо ефективний для інференсу моделей, де необхідно масштабувати короточасні обчислення при непередбачуваних навантаженнях. У таких архітектурах автоматизація досягається через інтеграцію з подієвими шинами (event-driven architecture) і механізмами автоматичного масштабування функцій.

Моніторинг моделей у виробничому середовищі здійснюється через телеметричні системи (Prometheus, Grafana, ELK-stack), які збирають метрики продуктивності, затримки, споживання пам'яті, точності прогнозів тощо.

Алгоритмічно такі системи використовують:

- методи кореляційного аналізу для виявлення деградації моделі;
- системи сповіщення на основі аномалій (Anomaly Detection);
- адаптивні тригери для автоматичного перезапуску тренування при виявленні погіршення результатів.

Це дозволяє побудувати замкнутий контур управління якістю моделей у реальному часі, що є основою для створення самонавчальних систем (Self-Adaptive AI Systems).

1.3 Аналіз проблем сучасних методів розгортання та підтримки моделей у хмарному середовищі

Розгортання та підтримка систем штучного інтелекту у хмарних середовищах сьогодні є складним багатокомпонентним процесом, який охоплює етапи інтеграції, масштабування, моніторингу, оновлення моделей, а також управління даними. Незважаючи на значні досягнення в галузі DevOps і MLOps, існує низка системних проблем, які впливають на ефективність, стабільність і відтворюваність таких систем.

Ці труднощі зумовлені як технічними обмеженнями інфраструктури, так і організаційними аспектами, що пов'язані з управлінням життєвим циклом моделі в розподілених обчислювальних середовищах.

Однією з найпоширеніших проблем сучасних систем розгортання ШІ-додатків є неможливість гарантовано відтворити результати навчання моделі, отримані на попередніх етапах.

Це виникає через низку факторів:

- невідповідність версій бібліотек та середовищ виконання;
- зміни у вихідних даних;
- відсутність централізованого сховища експериментів;

- людський фактор при оновленні коду або гіперпараметрів.

Наслідком є ситуація, коли дві ідентичні моделі, запуснені на різних вузлах хмарної інфраструктури, можуть давати відмінні результати прогнозування. Такий ефект називають “drift reproducibility”, і він призводить до зниження надійності всієї системи.

Для мінімізації цієї проблеми активно застосовуються підходи контейнеризації середовищ (Docker, Podman) та системи керування експериментами (MLflow, DVC, Weights & Biases). Вони дозволяють фіксувати конкретну конфігурацію програмного середовища, версії бібліотек, набори даних і результати навчання. Проте навіть ці інструменти не завжди гарантують повну відтворюваність — наприклад, при виконанні на GPU або TPU через відмінності в обчисленнях з плаваючою комою.

Наприклад, у 2022 році компанія OpenAI повідомила про труднощі відтворення поведінки мовних моделей GPT при оновленні CUDA-драйверів на нових вузлах кластеру. Незначні зміни у математичних бібліотеках призводили до відмінностей у результатах, що змусило команду створити власну систему контролю середовищ з фіксацією обчислювальних ядер та бібліотек до кожної версії моделі.

Таким чином, одним із напрямів удосконалення є створення уніфікованої моделі зберігання та ідентифікації середовищ виконання, яка б враховувала як програмні, так і апаратні залежності.

Також моделі штучного інтелекту залежать від якості та стабільності даних, тому зміна структури або статистичних властивостей вхідних наборів без належного контролю призводить до деградації точності моделі. Це явище відоме як data drift або concept drift.

У хмарних середовищах, де дані надходять потоково або з розподілених джерел, виявлення drift'ів ускладнюється через масштабність і динаміку. Існуючі інструменти, такі як Evidently AI чи TensorFlow Data Validation, дозволяють виявляти зміни у даних за допомогою статистичних методів

(наприклад, Kolmogorov–Smirnov test, Jensen-Shannon divergence), але їх інтеграція в CI/CD конвеєри часто є фрагментованою та не стандартизованою.

Ще одна проблема — зміна формату або структури даних у зовнішніх системах, що автоматично призводить до збоїв у пайплайнах обробки. У таких випадках втрачається узгодженість між версіями даних і відповідними моделями.

Шляхи вирішення:

- i) Впровадження Data Version Control (DVC) або аналогічних систем для синхронного контролю даних і моделей.
- ii) Розробка алгоритмів адаптивного перенавчання (Adaptive Retraining), які автоматично оновлюють модель при виявленні статистично значущих змін у даних.
- iii) Використання потокових аналітичних систем (Kafka Streams, Apache Flink), що дозволяють реагувати на зміни вхідних потоків у реальному часі.

Також розгортання великих нейронних мереж (наприклад, трансформерів із мільярдами параметрів) у хмарі супроводжується складністю масштабування обчислювальних ресурсів.

Хоча сучасні оркестратори (Kubernetes, OpenShift) забезпечують автоматичний розподіл навантаження, вони часто не враховують специфіку ШІ-обчислень, зокрема:

- нерівномірність споживання пам'яті під час тренування;
- різну тривалість обчислень у паралельних потоках;
- синхронізацію градієнтів між вузлами під час навчання (особливо при використанні GPU/TPU кластерів).

У результаті виникають “вузькі місця” (bottlenecks) — ресурси простоюють або, навпаки, перевантажуються.

Для прикладу, у великих системах типу Hugging Face Inference Endpoints масштабування моделей відбувається через попередньо створені контейнери,

але це збільшує час “холодного старту” (cold start latency), що може досягати кількох секунд — критично для real-time застосувань.

Можливі рішення:

- Використання динамічних механізмів “elastic scaling”, які прогнозують навантаження за допомогою моделей машинного навчання (наприклад, AWS Auto Scaling з ML-профілями).
- Оптимізація моделей через квантизацію, прунінг та дистиляцію, що дозволяє зменшити споживання ресурсів без втрати точності.
- Впровадження асинхронного паралелізму (Hogwild!, asynchronous SGD), який підвищує ефективність використання GPU-пулів.

Одним із найгостріших питань залишається захист моделей і даних, особливо у розподілених або публічних хмарних середовищах.

Типові ризики включають:

- несанкціонований доступ до моделей (модельна крадіжка або model inversion attacks);
- витік навчальних даних;
- впровадження шкідливого коду під час оновлення (supply-chain attacks).

У 2023 році компанія Microsoft зафіксувала випадок, коли у відкритому сховищі моделей було розміщено заражені версії Python-бібліотек, що використовувалися у пайплайнах навчання. Це показало вразливість CI/CD конвеєрів без контролю підписів артефактів.

Щоб боротися з цією проблемою, можна використовувати такі методи захисту:

- використання цифрових підписів контейнерів і моделей (Cosign, Notary);
- впровадження шифрування моделей у пам’яті (in-memory encryption) під час виконання;

- застосування конфіденційних обчислень (Confidential Computing) — технології, що забезпечують виконання обчислень у захищеному середовищі (наприклад, Intel SGX).

Навіть після успішного розгортання модель може втрачати точність унаслідок зміни реальних даних, нових патернів користувацької поведінки або накопичення шуму.

Більшість систем моніторингу фокусуються на технічних метриках (CPU, RAM, latency), але не відстежують якість прогнозів (model performance monitoring).

Для вирішення цього завдання потрібне впровадження систем комплексного моніторингу, які поєднують:

- технічні метрики (споживання ресурсів, затримки);
- аналітичні метрики (точність, recall, precision, F1-score);
- поведінкові показники користувачів (наприклад, зниження конверсії в бізнес-додатках).

Наприклад, під час розгортання моделі рекомендацій у системі Netflix виявили, що модель, оптимізована для короткострокової точності (watch-next), з часом втратила релевантність. Автоматичний моніторинг виявив відхилення метрик precision і recall, після чого система ініціювала автоматичне перенавчання на нових даних. Це зменшило час деградації якості з тижнів до годин.

ШІ-додатки зазвичай складаються з великої кількості компонентів — сховищ даних, пайплайнів обробки, моделей, API-шлюзів і аналітичних панелей. Кожен із них може працювати на різних технологічних стеках (Python, Java, Node.js, Go тощо).

Через це інтеграція та підтримка таких систем стає складною, особливо у великих мультихмарних середовищах (AWS + GCP + Azure).

Проблема полягає у відсутності єдиних стандартів опису інфраструктури та розгортання моделей.

Infrastructure as Code частково вирішує цю проблему (Terraform, Pulumi), але не завжди узгоджується з ML-конвеєрами, де потрібно одночасно керувати обчисленнями, даними, експериментами та моделями.

Способом удосконалення цього методу є створення уніфікованих описів ML-систем, які поєднують IaC, DataOps і MLOps у єдиній декларативній моделі — саме цей підхід поступово розвивається у сучасних фреймворках, таких як Kubeflow Pipelines, MLRun та Flyte.

Отже, попри активний розвиток інструментів автоматизації, сучасні методи розгортання ШІ-додатків у хмарному середовищі все ще мають низку проблем: від нестабільної відтворюваності та обмеженого контролю даних до питань безпеки й інтеграції.

Більшість існуючих рішень спрямовані на окремі аспекти — CI/CD, контейнеризацію чи моніторинг — тоді як ефективна хмарна система потребує комплексного, наскрізного підходу, який об'єднує всі ці рівні в єдину керовану архітектуру.

Саме тому актуальним є створення інноваційних методів автоматизації, що поєднують адаптивні алгоритми, динамічне масштабування, контроль версій та глибоку інтеграцію інструментів MLOps. Такий підхід дозволяє не лише підвищити ефективність розгортання ШІ-додатків, але й забезпечити їхню стійкість, безпечність і прогнозованість — що є основною практичною цінністю сучасних систем.

Висновки до розділу

У першому розділі магістерської роботи було здійснено ґрунтовний аналіз сучасних підходів до автоматизації розгортання систем штучного інтелекту в хмарному середовищі. Основна увага приділялася визначенню ролі хмарних технологій, вивченню актуальних методів автоматизації процесів розгортання та оцінці наявних проблем, які обмежують ефективність функціонування таких систем.

У розділі, присвяченому поняттю хмарних технологій та їх ролі у розгортанні штучного інтелекту, було з'ясовано, що саме хмарні платформи створюють базис для масштабованого та гнучкого розгортання інтелектуальних додатків. Використання моделей IaaS, PaaS і SaaS забезпечує оптимальний розподіл обчислювальних ресурсів, швидке розгортання компонентів системи, централізоване керування процесами обробки даних і зменшення вартості експлуатації. Такі можливості дають змогу інтегрувати складні алгоритми машинного навчання та глибокого навчання у стабільне середовище, що підтримує постійне оновлення та масштабування. Хмарні технології, таким чином, виступають каталізатором розвитку штучного інтелекту, забезпечуючи не лише технічну інфраструктуру, а й методологічну основу для побудови сучасних інтелектуальних сервісів.

Під час розгляду сучасних методів автоматизації та розгортання ШІ-додатків у хмарі було встановлено, що ефективна робота таких систем неможлива без інтеграції автоматизованих процесів життєвого циклу моделі. Особливу роль у цьому відіграє концепція MLOps, яка є логічним продовженням принципів DevOps і спрямована на підвищення надійності, відтворюваності та контрольованості процесів навчання і розгортання моделей. У межах MLOps передбачено автоматизацію таких етапів, як підготовка даних, тестування моделей, управління версіями, моніторинг результатів і повторне навчання у разі погіршення якості прогнозів. Застосування пайплайнів CI/CD, механізмів відстеження змін параметрів і використання модульних компонентів сприяє підвищенню стабільності системи та скороченню часу від розробки до впровадження. Разом із тим, виявлено, що більшість існуючих підходів орієнтовані на окремі частини процесу, тоді як повна наскрізна автоматизація життєвого циклу моделі залишається актуальною проблемою.

У розділі, присвяченому аналізу проблем сучасних методів розгортання та підтримки моделей у хмарному середовищі, було окреслено коло основних викликів, з якими стикаються розробники та дослідники. Серед них — труднощі з відтворюваністю моделей у різних середовищах, зниження точності

прогнозів унаслідок зміни структури або характеристик даних, а також обмеження масштабованості при роботі з великими нейронними мережами. Окрему увагу приділено питанням безпеки та конфіденційності даних, що є надзвичайно важливими у випадку хмарних розгортань. Проблемою залишається також складність інтеграції різних інструментів і технологічних стеків, оскільки сучасні ШІ-рішення зазвичай об'єднують модулі, створені на основі різних мов програмування, бібліотек і фреймворків. Аналіз практичного досвіду компаній, таких як OpenAI, Microsoft і Netflix, продемонстрував, що навіть за наявності потужних обчислювальних ресурсів та зрілих інженерних практик забезпечення стабільності й контрольованості моделей залишається складним завданням.

Попри зазначені труднощі, у роботі визначено шляхи подолання основних проблем, серед яких — уніфікація архітектури, впровадження стандартів відстеження версій моделей, удосконалення механізмів моніторингу, підвищення рівня автоматизації процесів оновлення та розгортання. Ці підходи є базовими передумовами для створення більш ефективної, гнучкої та безпечної системи управління життєвим циклом штучного інтелекту.

Узагальнюючи результати проведеного аналізу, можна стверджувати, що хмарні технології в поєднанні з принципами MLOps формують основу для побудови сучасних систем штучного інтелекту, здатних до масштабування, адаптації та автономного функціонування. Водночас існуючі методи автоматизації та підтримки моделей потребують подальшого вдосконалення для забезпечення повної інтеграції процесів розробки, розгортання та експлуатації. Це зумовлює необхідність створення нового підходу до організації архітектури таких систем, який би забезпечував високу ефективність, безпеку та надійність під час роботи у хмарному середовищі.

2 ДОСЛІДЖЕННЯ АРХІТЕКТУРНИХ РІШЕНЬ ТА ВИБІР ІНСТРУМЕНТІВ ДЛЯ АВТОМАТИЧНОГО РОЗГОРТАННЯ ШІ-СИСТЕМИ

2.1 Огляд інструментів контейнеризації та оркестрації

Сучасні системи штучного інтелекту функціонують у середовищі з високими вимогами до продуктивності, масштабованості та відтворюваності процесів. Наявність декількох взаємопов'язаних компонентів — сервісів інференсу, модулів тренування моделей, моніторингових підсистем та інтерфейсів керування — робить процес розгортання складним і чутливим до найменших змін у конфігурації середовища. Саме тому контейнеризація стала ключовим технологічним підходом, який забезпечує стабільність і прогнозованість функціонування таких систем незалежно від обчислювальної платформи.

У контексті цієї магістерської роботи контейнеризація виконує роль фундаментального шару, що гарантує стабільну роботу повністю реалізованої системи автоматизованого розгортання ШІ-застосунків, яка охоплює інференс моделей, їх тренування, моніторинг, CI/CD-конвеєр, blue-green оновлення та веб-інтерфейс. Кожен компонент системи функціонує у власному контейнері, що забезпечує чіткий розподіл обов'язків, можливість незалежного оновлення та високу керованість життєвим циклом компонентів.

Контейнеризація передбачає розгортання застосунку в ізольованому середовищі, яке містить необхідний набір бібліотек, залежностей і конфігурацій [12]. Такий підхід вирішує проблему відмінностей між продуктивним, тестовим і локальним середовищем, які особливо критичні для машинного навчання, де зміна версії бібліотеки або драйвера може суттєво вплинути на результат роботи моделі.

Архітектура контейнера в ML-проектах традиційно включає:

- базовий образ із Python або іншою мовою виконання;
- встановлені бібліотеки машинного навчання, такі як PyTorch [16];
- драйвери та залежності для GPU-підтримки;
- власний код сервісу або моделі;
- конфігураційні файли та середовищні параметри.

Типова схема життєвого циклу контейнера виглядає так, як зображено на Рисунку 1.

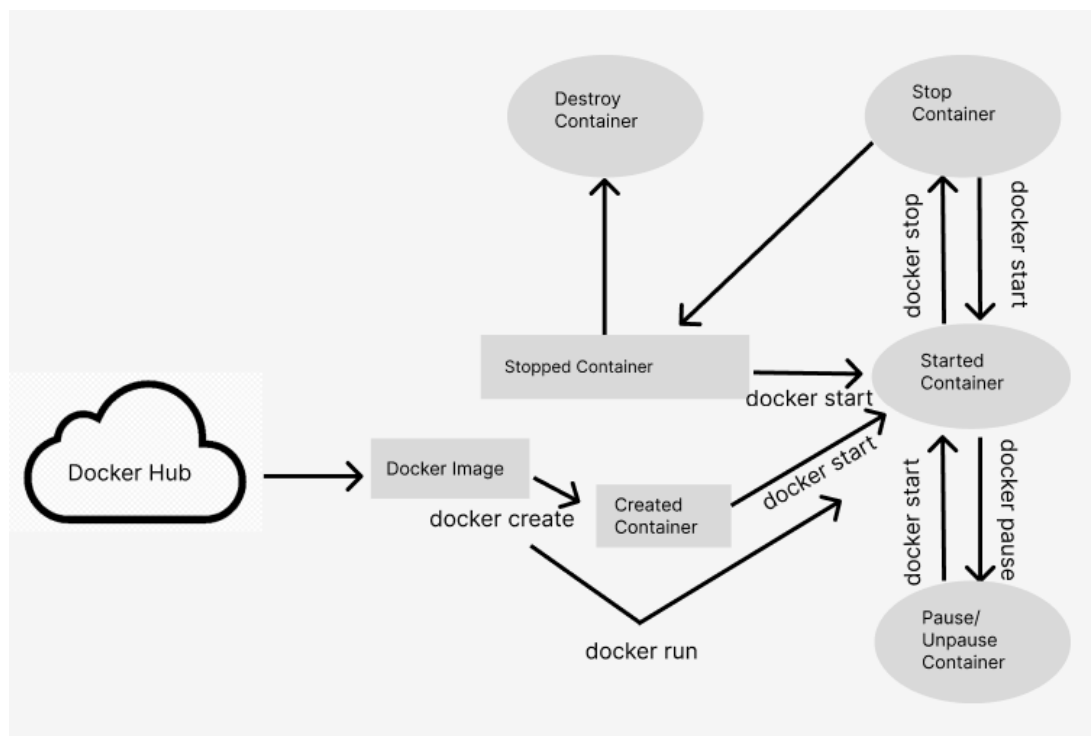


Рисунок 1 - Схема життєвого циклу Docker-контейнерів

У реалізованій системі контейнеризація дозволяє досягти повної відтворюваності сервісів. Усі вони функціонують як автономні, але інтегровані між собою мікросервіси.

Серед усіх доступних технологій контейнеризації найбільш поширеним і розвиненим є Docker. Його архітектура передбачає наявність центрального демона Docker Engine, який відповідає за побудову образів, запуск контейнерів, керування мережею та інтеграцію з апаратними прискорювачами. Такий підхід забезпечує високу продуктивність і відмінну підтримку GPU [5].

У системах машинного навчання підтримка GPU є критично важливою. Docker забезпечує повну сумісність із NVIDIA Container Toolkit [13], що надає можливість виконувати навчання нейронної модулі на GPU без зміни інфраструктури сервера. Docker автоматично інтегрує змінні середовища та

драйвери CUDA, а також гарантує сумісність версій PyTorch із відповідними бібліотеками [5].

Важливою є й сумісність Docker з DevOps-процесами. Побудова образів у CI/CD-конвеєрі відбувається швидко, а Docker Hub або приватний реєстр забезпечують централізоване зберігання артефактів системи. Саме тому у реалізованій системі всі компоненти будуються як окремі Docker-образи, що дозволяє легко керувати залежностями та версіями.

Docker має низку переваг, серед яких стабільність, передбачуваність і універсальність. Він є оптимальним вибором для масштабованих ML-рішень, оскільки дозволяє швидко оновлювати моделі, тестувати різні середовища навчання та розгортати сервіси як локально, так і у хмарі [5].

Попри потребу у демоні, що може виглядати як недолік у деяких сценаріях, у MLOps-проектах цей аспект не є критичним, оскільки важливішими стають стабільність роботи та інтеграція з GPU.

Хоча Docker є домінуючим інструментом, інші технології контейнеризації також заслуговують на увагу.

Podman приваблює бездемонною архітектурою та можливістю запуску контейнерів у режимі “rootless”. У середовищах з високими вимогами до безпеки Podman розглядається як більш безпечна альтернатива Docker. Проте підтримка GPU у Podman є менш стабільною, ніж у Docker. Це обмежує його використання у ML-проектах, де апаратні прискорювачі відіграють ключову роль. Крім того, Podman має менш розвинену екосистему для CI/CD і не пропонує такого самого рівня підтримки інструментів MLOps, як Docker.

LXC/LXD — це система контейнерів системного рівня, орієнтована на запуск повноцінних операційних систем у контейнеризованому форматі. Вона забезпечує ширші можливості у сфері інфраструктурної віртуалізації, ніж Docker, проте є надлишковою для мікросервісних ML-систем. Відсутність нативної інтеграції з NVIDIA Runtime і громіздкість керування роблять LXC/LXD менш придатними для модульних MLOps-рішень.

З огляду на специфіку системи, що розглядається в цій роботі, Docker став найбільш оптимальним вибором, оскільки забезпечив GPU-прискорене навчання, кероване інференс-середовище та повноцінну інтеграцію з CI/CD.

Контейнеризація вирішує проблему ізоляції та відтворюваності, проте не вирішує питання автоматичного контролю стану, масштабування або оновлення сервісів. У складних ІІТ-системах, де одночасно працює декілька контейнерів, важливо мати інструмент оркестрації, який забезпечує керування їхнім життєвим циклом.

У загальному випадку оркестрація охоплює управління розгортанням, балансуванням навантаження, моніторингом стану, автоматичним відновленням після збою та оновленням за стратегіями, які зменшують або повністю усувають простій системи.

Kubernetes є промисловим стандартом для оркестрації контейнерів. Його складна, але гнучка архітектура забезпечує можливість побудови величезних розподілених систем із автоматичним масштабуванням, самовідновленням та потужною конфігурацією мережових політик. Схематично базова архітектура Kubernetes подана на Рисунку 2.

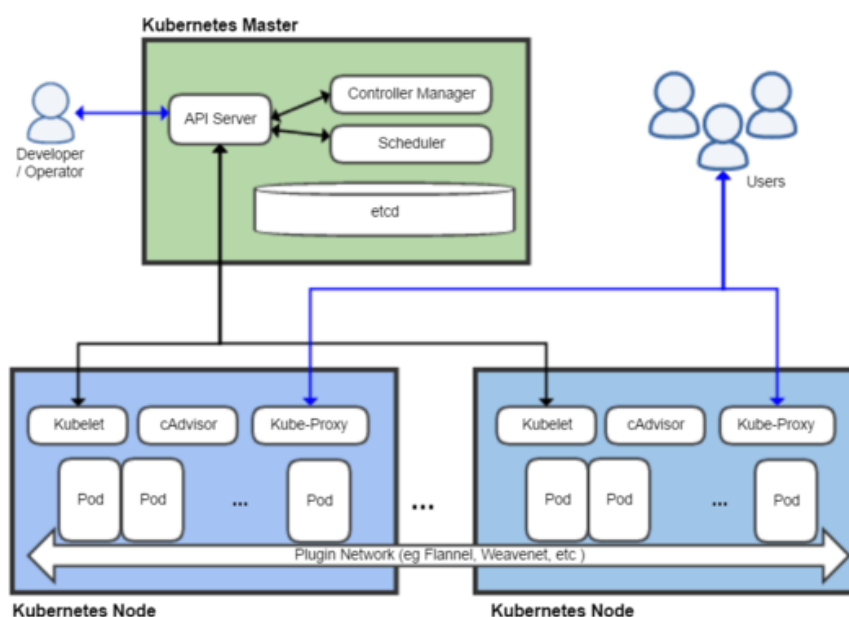


Рисунок 2 - Базова архітектура Kubernetes

Його архітектура орієнтована на роботу з великими та розподіленими мікросервісними системами, де необхідно забезпечити високу доступність, стійкість до збоїв та можливість автономного реагування на зміну навантаження. Kubernetes виступає своєрідною операційною системою для кластерів, абстрагуючи фізичне чи віртуальне апаратне забезпечення та надаючи уніфікований шар керування контейнерами.

У своїй основі Kubernetes реалізує декларативний підхід до управління, коли користувач описує бажаний стан системи у вигляді YAML-конфігурацій, а саме середовище автоматично підтримує цей стан, реагуючи на будь-які відхилення. Такий механізм забезпечується контролерами та планувальниками, які постійно перевіряють поточний стан кластеру та узгоджують його із заданою конфігурацією. Завдяки цьому Kubernetes може автоматично створювати нові екземпляри сервісів, перезапускати контейнери, переміщувати їх на інші вузли та масштабувати ресурси залежно від потреб розробленої системи.

Архітектурно система складається з керувального рівня та робочих вузлів. Керувальний рівень включає сервер API, який є центральною точкою взаємодії з кластером, планувальник, що визначає розміщення контейнерів, та набір контролерів, які стежать за відповідністю станів. Робочі вузли виконують безпосереднє навантаження: на них розміщуються підсистеми, що складаються з одного чи декількох контейнерів, що пов'язані внутрішньою логікою.

Однією з ключових переваг Kubernetes є його здатність управляти мережевою взаємодією між сервісами без необхідності ручної конфігурації. Система створює власний мережевий шар, де кожен контейнер отримує власну IP-адресу, а взаємодія здійснюється через внутрішні DNS-імена. Усі сервіси автоматично реєструються, а трафік балансовано розподіляється між їх екземплярами. Цей принцип робить Kubernetes зручним для розподілених III-систем, де окремі модулі повинні ефективно комунікувати один з одним у кластерному середовищі.

Система також передбачає декілька механізмів масштабування, серед яких горизонтальне масштабування на основі навантаження, вертикальне

масштабування ресурсів та автоматичне під'єднання додаткових вузлів. Це дозволяє адаптувати продуктивність сервісів у реальному часі залежно від інтенсивності запитів або дедалі зростаючих обчислювальних потреб моделей машинного навчання. Разом з механізмом самовідновлення Kubernetes здатний автоматично реагувати на збої, піднімаючи нові екземпляри при падінні попередніх або переміщуючи їх у разі несправності окремих вузлів.

У роботі з даними Kubernetes пропонує механізми стійких томів, що дозволяють зберігати моделі, проміжні артефакти або логи навіть після перезавантаження контейнерів. Це критично важливо у випадку навчання моделей, де результати виконання мають зберігатися незалежно від стану обчислювальних вузлів.

З точки зору MLOps-практик Kubernetes є універсальною платформою, оскільки може інтегруватися з інструментами автоматичного тренування моделей, такими як Kubeflow, MLRun або Flyte [2]. Ці системи використовують Kubernetes як базову інфраструктуру для створення складних конвеєрів, у межах яких плануються задачі тренування, валідації, деплойменту та моніторингу моделей. Таке поєднання дозволяє формувати замкнуті цикли автоматичного оновлення моделей, що є важливою умовою побудови автоматизованих ШІ-рішень.

Також Kubernetes надає можливість ефективно працювати у різних середовищах. Кластери можуть бути розгорнуті у публічних хмарах, у приватних дата-центрах або у гібридних конфігураціях, що забезпечує незалежність від провайдера та високу міграційну гнучкість. Це робить платформу придатною для довготривалих інфраструктурних рішень, які потребують стабільності, масштабованості та контрольованості.

Попри численні переваги, Kubernetes має і свої недоліки. Основним є його складність: налаштування, обслуговування та експлуатація кластеру вимагають глибоких технічних знань і значної інфраструктурної підтримки. Для невеликих або помірних за масштабом проєктів Kubernetes може виявитися надлишковим, адже організація кластеру нерідко потребує більше ресурсів, ніж сам застосунок.

Проте в масштабних ІІІ-платформах, де необхідна динамічна оркестрація обчислень, цей інструмент є одним з найкращих.

Таким чином, Kubernetes є потужним, всеосяжним рішенням для керування контейнеризованими застосунками у розподілених середовищах. Його архітектура, базована на декларативному підході, механізми автоматичного відновлення та масштабування, а також широка екосистема інструментів роблять його ключовою технологією у побудові сучасних інтелектуальних систем.

Також варто розглянути Docker Compose. Це інструмент, який істотно розширює можливості контейнеризації та дозволяє переходити від індивідуального керування окремими контейнерами до комплексного управління багатокомпонентними системами. На відміну від базового створення та запуску одного контейнера, Docker Compose забезпечує опис та одночасне розгортання цілих стеків сервісів у вигляді єдиного узгодженого середовища [4]. Завдяки цьому Compose є ключовим інструментом у побудові мікросервісної архітектури та застосовується повсюдно у DevOps- і MLOps-процесах.

Основна ідея Docker Compose полягає в тому, що конфігурація всієї системи зберігається у декларативному файлі. У цьому файлі визначаються окремі сервіси — такі як веб-застосунок, база даних, кеш, брокер повідомлень чи сервіс машинного навчання — разом з усіма параметрами їхнього запуску. Compose об'єднує ці сервіси у єдину логічну систему, автоматично створює мережі для їхньої взаємодії, монтує томи з даними, керує залежностями та забезпечує контрольований запуск у правильній послідовності [4].

Архітектура Docker Compose побудована таким чином, щоб відокремити застосунок від інфраструктурних деталей. Розробник описує лише структуру сервісів, зазначає, які образи необхідно використовувати, які порти потрібно пробросити, які змінні середовища встановити та які томи змонтувати. Усі ці параметри об'єднуються у єдине середовище виконання, незалежне від конфігурації локального сервера чи хмарної машини. Це означає, що складні системи можуть бути розгорнуті буквально однією командою, що значно

прискорює DevOps-процеси та знижує ризик помилок, пов'язаних із ручною конфігурацією.

Для систем штучного інтелекту Docker Compose має особливу цінність, оскільки такі системи зазвичай складаються з декількох окремих модулів. Наприклад, сервіс інференсу, модуль тренування, сервіс моніторингу, панель аналітики та веб-клієнт — усі вони можуть працювати як автономні контейнери, але повинні комунікувати між собою у спільній мережевій інфраструктурі. Compose надає можливість створити для цього ізольовану мережу, в межах якої кожен сервіс отримує власне DNS-ім'я, а їх взаємодія відбувається через внутрішні порти. Таким чином усувається необхідність вручну керувати мережевими параметрами або використовувати додаткові проксі-шари.

Додатковою перевагою Compose є підтримка профілів. Завдяки цьому механізму можна описувати різні варіанти розгортання однієї системи, активуючи лише ті сервіси, які потрібні в конкретному сценарії. У складених ML-системах це дозволяє відокремити, наприклад, звичайний режим роботи, режим оновлення моделі або режим тестування.

Compose значно спрощує керування томами, які необхідні у проєктах машинного навчання для зберігання моделей, логів та проміжних результатів тренування. Монтування каталогів у вигляді томів гарантує, що дані не будуть втрачені при перезавантаженні контейнера або при його оновленні. Це дає можливість реалізувати механізми відтворюваності навчання моделей та надійне зберігання їхніх артефактів.

Важливою рисою Docker Compose є можливість інтеграції з CI/CD-конвеєрами. У поєднанні з GitHub Actions або іншими DevOps-інструментами Compose дозволяє повністю автоматизувати процес побудови, тестування та розгортання системи [7].

Контейнеризовані системи, керовані через Docker Compose, характеризуються високим рівнем автономності [4]. Кожен сервіс має власні ресурси, власну мережеву конфігурацію та власний життєвий цикл, що дозволяє

легше виявляти помилки, швидше оновлювати компоненти та масштабувати їх у разі необхідності.

Загалом Docker Compose є оптимальним рішенням для систем середнього рівня складності, особливо таких, що орієнтовані на машинне навчання, де необхідна одночасна робота декількох сервісів із різними функціями. Він забезпечує високу гнучкість, легкість розгортання та контроль над конфігурацією, що робить його одним із ключових технологічних інструментів у побудові повноцінних MLOps-рішень.

Docker Swarm є природним продовженням Docker Compose. Він пропонує мінімалістичний підхід до оркестрації. Його архітектура проста, а розгортання займає лічені хвилини. Проте можливості Swarm значно поступаються Kubernetes, а розвиток цього інструменту суттєво уповільнився.

У своїй основі Swarm побудований на моделі кластеру, який складається з керувальних вузлів і робочих вузлів. Керувальні вузли відповідають за прийняття рішень про розміщення сервісів, підтримку стану кластера та синхронізацію інформації між учасниками через консенсус-алгоритм Raft. Робочі вузли виконують контейнеризовані задачі, і їх кількість може змінюватись динамічно залежно від поточних потреб. Завдяки такій структурі Swarm забезпечує базову стійкість до відмов і може автоматично переміщувати задачі у випадку недоступності окремих вузлів.

Однією з ключових характеристик Docker Swarm є його простота. Для розгортання кластера достатньо виконати декілька команд, після чого вузли автоматично приєднуються до кластеру та синхронізують свій стан. Це робить Swarm доступним для розробників, які не мають глибокого досвіду в управлінні розподіленими системами. Синтаксис YAML-файлів та керування сервісами у Swarm багато в чому нагадує Docker Compose, а тому перехід від локального розгортання до кластерного проходить без суттєвої перебудови системи.

У сфері оркестрації Swarm підтримує основні механізми, характерні для хмарних систем: балансування навантаження між екземплярами сервісів, автоматичне відновлення контейнерів після збою, відстеження стану задач у

режимі реального часу та можливість горизонтального масштабування. Кожен сервіс описується як абстракція, яка може містити декілька екземплярів, що розподіляються між доступними вузлами. Swarm автоматично забезпечує рівномірний розподіл контейнерів та перенесення їх у разі зміни доступності ресурсів.

Ще однією важливою особливістю Docker Swarm є його власна мережа оверлейного типу. Вона дозволяє екземплярам сервісів, розміщеним на різних вузлах, взаємодіяти так само, як і локальні контейнери у Docker Compose. Сервіси отримують власні DNS-імена, а маршрутизація запитів здійснюється внутрішньою системою балансування. Такий підхід значно спрощує роботу з мікросервісними застосунками, які можуть бути розділені між кількома фізичними або віртуальними серверами.

Також важливо зазначити, що Swarm забезпечує вбудовану підтримку стратегій оновлення сервісів, включно з оновленням із мінімальним збоєм роботи. Система надає можливість поступово оновлювати екземпляри контейнерів, контролюючи їхній стан та можливість повернення до попередньої версії у разі виникнення помилок. Хоча функціональність Swarm у цій сфері не така гнучка, як у Kubernetes, для багатьох застосунків вона є цілком достатньою.

Docker Swarm також має певні обмеження, які насамперед пов'язані з його невеликою екосистемою та менш активним розвитком спільноти. У порівнянні з Kubernetes він пропонує суттєво менше механізмів для складних сценаріїв оркестрації, не має розвинених інструментів для побудови складних DevOps- та MLOps-конвеєрів і не забезпечує достатньо глибокої інтеграції з GPU-вузлами або спеціалізованими ML-фреймворками. Проте його простота та легкість конфігурації роблять Swarm привабливим для систем середнього рівня складності, а також для середовищ, де важливо зберегти максимальну схожість між локальним розробницьким середовищем і кластерною інфраструктурою.

У підсумку Docker Swarm можна охарактеризувати як мінімалістичний, інтуїтивний і тісно інтегрований з Docker механізм оркестрації, який забезпечує базові можливості побудови високодоступних і розподілених систем. Хоча він

поступається Kubernetes масштабованістю та функціональністю, Swarm залишається цінним рішенням для розгортань, де важливими є простота, швидкість налаштування та сумісність з екосистемою Docker.

Розглянемо ще одну платформу для оркестрації HashiCorp Nomad. Це універсальна та легковагова платформа для оркестрації робочих навантажень, що дозволяє розміщувати, планувати та управляти контейнеризованими й неконтейнеризованими застосунками в розподіленому середовищі. На відміну від багатьох інших оркестраторів, Nomad не обмежується контейнерами та може працювати з системними процесами, виконуваними файлами, Java-додатками й Docker-контейнерами, що робить його більш гнучким інструментом. Така універсальність забезпечує можливість використання Nomad у змішаних середовищах, де потрібно одночасно керувати як традиційним програмним забезпеченням, так і мікросервісами.

Архітектура Nomad побудована на принципах максимальної простоти та одновузлової легкості розгортання. Система складається з агентів, які можуть функціонувати у двох режимах — серверному та клієнтському. Сервери відповідають за зберігання глобального стану, планування задач та координацію роботи кластера, а клієнтські вузли виконують завдання, розподілені планувальником. Ці компоненти взаємодіють через кластери, використовуючи внутрішній механізм консенсусу Raft, який гарантує узгодженість станів навіть у разі часткових збоїв. Nomad передбачає високу стійкість до відмов, а проста архітектура забезпечує швидке відновлення та мінімальну кількість рухомих частин у системі.

Однією з ключових особливостей Nomad є його гнучка система планувальників. Він підтримує різні типи планувань, зокрема періодичні, довготривалі або пакетні задачі, що робить його зручним не лише для постійно працюючих сервісів, а й для задач, що пов'язані з машинним навчанням.

Nomad пропонує високопродуктивний, компактний і модульний підхід до мережевої взаємодії. На відміну від Kubernetes, який використовує складну систему абстракцій для взаємодії між сервісами, Nomad реалізує більш простий

мережевий шар. При цьому Nomad часто поєднують із Consul — ще одним продуктом HashiCorp — який відповідає за виявлення сервісів та розподілену конфігурацію. Взаємодія Nomad і Consul формує повноцінне середовище для мікросервісів, забезпечуючи автоматичну реєстрацію сервісів, внутрішню маршрутизацію та захищений обмін даними між вузлами. У деяких сценаріях така зв'язка може бути альтернативою Kubernetes, особливо якщо система не вимагає складних політик масштабування або розвинених мережових конфігурацій.

Щодо керованості, то Nomad виділяється надзвичайно простою декларативною моделлю опису задач. Конфігурації формуються у вигляді HCL (HashiCorp Configuration Language), що поєднує зручність читання та висловлення технічних параметрів. В описі задач визначаються ресурси, залежності, драйвери виконання, томи даних та інші параметри. Завдяки цьому розробник отримує повний контроль над розгортанням застосунків, не стикаючись зі складністю, характерною для Kubernetes. Це робить Nomad привабливим рішенням для команд, для яких важлива передбачуваність та мінімалізм у керуванні інфраструктурою.

Незважаючи на свої переваги, даний інструмент має й певні обмеження. Хоча Nomad й підтримує контейнеризацію через Docker, його інтеграція з GPU-прискорювачами менш розвинена, ніж у Kubernetes або Docker з NVIDIA Runtime. Крім того, екосистема навколо Nomad не така велика, що може ускладнювати розробку комплексних ML-конвеєрів, які потребують широкої палітри спеціалізованих модулів і операцій. Сам Nomad залишається чудовим планувальником, проте його функціональність часто доповнюється іншими інструментами, такими як Consul і Vault, що може ускладнити загальну архітектуру.

У підсумку можна сказати, що HashiCorp Nomad є сильним і гнучким інструментом для оркестрації змішаних типів навантажень, здатним забезпечити високу продуктивність, стабільність і простоту. Його універсальність, модульність та лаконічна архітектура роблять його привабливим для систем, де необхідно водночас керувати традиційними сервісами та контейнерами. Проте у

контексті систем машинного навчання, що вимагають глибокої інтеграції з GPU, масштабних автоматизованих пайплайнів та спеціалізованих ML-інструментів, Nomad часто поступається Kubernetes та інструментам Docker. Незважаючи на це, він залишається цінним елементом екосистеми оркестраторів завдяки своїй простоті, швидкодії та здатності ефективно працювати у великих розподілених середовищах.

З огляду на технічні вимоги до системи — автоматичне розгортання моделей, безперервне оновлення сервісу інференсу, наявність GPU-прискореного тренування та окремих служб моніторингу й інтерфейсу — найбільш логічним рішенням стало використання Docker Compose у поєднанні зі стратегією blue–green deployment.

Таке поєднання забезпечує:

- відсутність простоїв під час оновлення моделі;
- чітку ізоляцію версій сервісів інференсу (blue і green);
- можливість тестувати нову версію без впливу на поточну;
- простоту налаштування та експлуатації;
- повну сумісність із CI/CD через GitHub Actions;
- мінімальні вимоги до інфраструктури при збереженні професійного рівня автоматизації.

Особливістю реалізованої системи є те, що blue та green середовища працюють паралельно, причому перехід між ними здійснюється автоматично після health-check від нового стеку. Це гарантує безпечний деплой та дає можливість миттєво повернутися до попередньої версії у разі потреби.

Таким чином, обрана архітектура оркестрації повністю відповідає вимогам сучасних систем штучного інтелекту та рівню складності реалізованого проєкту, забезпечуючи надійність, автономність та передбачуваність роботи всіх компонентів.

2.2 Тренування нейромережових моделей у контейнеризованому середовищі

Тренування моделей глибокого навчання є однією з найресурсомісткіших та найскладніших частин розробки штучного інтелекту. Цей етап характеризується високими вимогами до обчислювальної потужності, стабільності середовища, відтворюваності експериментів та контролю за залежностями. Традиційний підхід до організації ML-експериментів, що базується на ручному налаштуванні бібліотек і середовищ, нерідко призводить до помилок несумісності, різних результатів на різних серверах та труднощів у перенесенні моделі з етапу дослідження до етапу експлуатації. Ці проблеми стають особливо помітними під час роботи з GPU-прискорювачами, драйверами NVIDIA та оновлюваними бібліотеками лінійної алгебри [8].

Саме тому контейнеризація стала обов'язковою частиною сучасних MLOps-підходів, оскільки забезпечує стандартизоване, ізольоване та відтворюване середовище виконання. У контейнеризованому середовищі тренування нейромережових моделей відбувається в умовах, коли усі залежності чітко визначені та закріплені, а зміни в конфігурації не впливають на інші частини системи. Це дозволяє створити стабільну інфраструктуру для роботи з моделями, незалежно від того, чи виконується тренування локально, у хмарі або на окремому GPU-сервері (рис. 3).

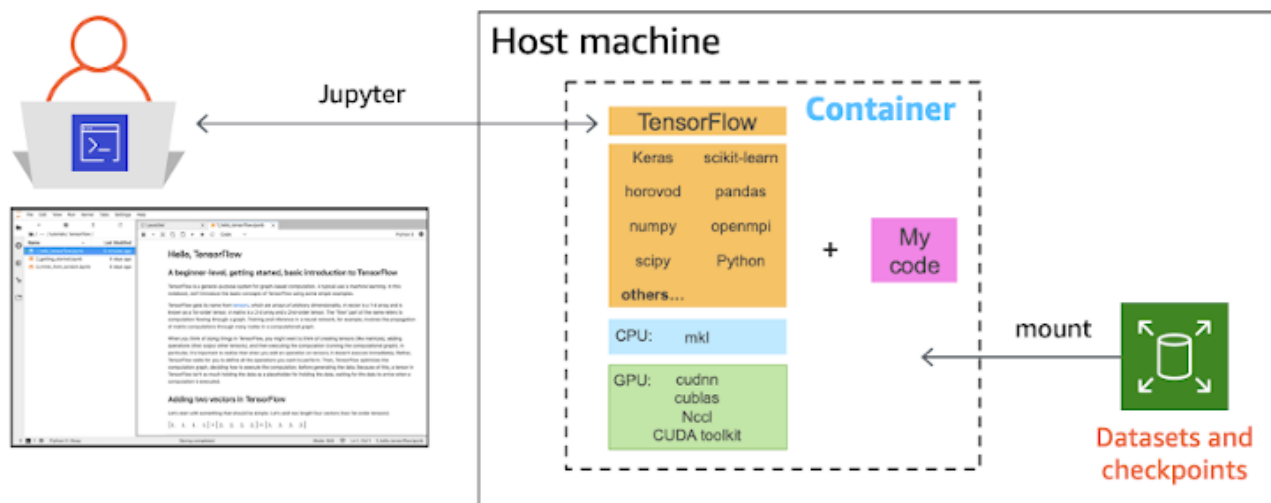


Рисунок 3 - Приклад контейнеризованої інфраструктури для роботи з моделями

Контейнеризоване середовище для тренування моделей характеризується тим, що воно може включати у себе всі необхідні компоненти: інтерпретатор Python [15], бібліотек для навчання нейромереж, драйвери CUDA, системні залежності та власне навчальні скрипти. Таким чином, розробник отримує повністю керовану та ізольовану інфраструктуру для експериментів. Однією з ключових переваг цього підходу є гарантія того, що одна й та сама модель, навчена в одному контейнері, буде мати ідентичні результати на іншому сервері, за умови однакового набору ресурсів. Це забезпечує відтворюваність, яка є фундаментальною вимогою до наукових досліджень та промислових ML-систем.

У сучасних системах штучного інтелекту найбільш поширеними інструментами для розробки та навчання нейромережеских моделей є бібліотеки PyTorch та TensorFlow. Ці бібліотеки пропонують повноцінний набір засобів для роботи з тензорами, тренування моделей, обчислень на GPU, серіалізації нейронних архітектур та розгортання моделей у виробничих середовищах. Попри це, підходи до роботи з моделями, їх внутрішня архітектура та філософія використання суттєво відрізняються. Саме тому аналіз цих бібліотек є важливим для обґрунтованого вибору конкретного ML-фреймворку у реалізованій системі автоматизованого контейнеризованого розгортання.

PyTorch позиціонується як гнучка та інтерактивна платформа для створення нейромережеских моделей. Основою його архітектури є механізм динамічного обчислювального графа. На відміну від статичних графів, динамічні графи формуються в момент виконання, тому структура моделі може змінюватися залежно від вхідних даних чи логіки роботи. Такий підхід особливо зручний для наукового моделювання, створення складних або нетипових архітектур, а також для сценаріїв, де модель повинна реагувати на умовні гілки або нерегулярні дані [16].

PyTorch має тісну інтеграцію з CUDA і надає розробнику можливість у будь-який момент перемикаєти моделі та дані між CPU та GPU [16]. Автоматичне диференціювання реалізовано через механізм Autograd, який дозволяє обчислювати градієнти без жодних додаткових дій з боку розробника.

Окремої уваги заслуговує екосистема PyTorch. Модуль torchvision включає переднавчені моделі, архітектури ResNet, MobileNet, VGG тощо, а також популярні набори даних і перетворення для задач комп'ютерного зору. PyTorch також підтримує TorchScript — механізм, що перетворює динамічні моделі у статичні графи для пришвидшення виконання та оптимізації під продакшн-середовище [17].

Завдяки простому API, природній інтеграції з Python, легкості налагодження й високій прозорості процесів PyTorch став домінуючим інструментом у дослідницьких спільнотах, стартапах та багатьох промислових MLOps-рішеннях [16].

Щодо TensorFlow, то даний інструмент є однією з найдавніших і наймасштабніших платформ для глибокого навчання, розроблених компанією Google. У його архітектурі першочергово закладено статичний обчислювальний граф, який оптимізується перед виконанням. Такий підхід забезпечує високу ефективність, особливо при тренуванні великих моделей або розгортанні інференсу на високонавантажених сервісах.

TensorFlow також має тісну інтеграцію з екосистемою Google, підтримує апаратні прискорювачі TPU та включає TensorFlow Serving — виробничу платформу для інференсу, яка дозволяє розгортати моделі у вигляді масштабованих сервісів. Завдяки TensorFlow Lite можливе розгортання моделей на мобільних пристроях та вбудованих системах.

Крім базового API, TensorFlow включає Keras — високорівневий інтерфейс, який значно спрощує створення нейромереж. Keras забезпечує модульність, читабельність коду та можливість швидкої побудови архітектур без занурення у низькорівневі деталі. Водночас TensorFlow має складнішу систему конфігурацій, а його статична модель обчислень може бути менш зручною для досліджень, де важливо мати динамічний контроль над параметрами моделі.

TensorFlow широко використовується у великих компаніях завдяки своїй масштабованості, зрілому продакшн-середовищу та можливостям оптимізації під різні обчислювальні архітектури.

Попри схожість у функціональному наповненні, PyTorch і TensorFlow пропонують різні моделі роботи. PyTorch забезпечує природну інтеграцію із середовищем виконання Python [16]. TensorFlow, своєю чергою, орієнтований на оптимізоване виконання у вигляді статичних графів, що особливо ефективно при масштабованому інференсі.

PyTorch вважається більш зручним для досліджень і швидкої побудови прототипів моделей, оскільки дозволяє динамічно змінювати структуру мережі в процесі виконання [16]. Це робить його більш гнучким. У свою чергу TensorFlow пропонує потужну екосистему для продакшн-середовища, включно з інструментами для розгортання, моніторингу та оптимізації моделей, проте є складнішим у початковому освоєнні.

Щодо продуктивності, TensorFlow традиційно вважається більш оптимізованим для великих моделей у статичному виконанні, тоді як PyTorch пропонує більш плавний досвід у динамічних сценаріях та має високу ефективність завдяки оптимізаціям CUDA і TorchScript.

У контексті MLOps-систем обидві бібліотеки можуть бути інтегровані у контейнеризоване середовище [18], проте саме PyTorch значно простіше поєднується з Docker і зберігає менше прихованих залежностей [16], що забезпечує більш передбачуване середовище для тренування та інференсу.

У процесі тренування моделей важливим аспектом також є робота з графічними процесорами. Використання GPU-прискорення дозволяє скоротити час навчання нейромереж у десятки та сотні разів порівняно з CPU [8].

Контейнеризація, з одного боку, створює ізольоване середовище для нейромережевих моделей, а з іншого — ускладнює доступ застосунків до апаратних ресурсів хоста. Стандартна модель Docker за замовчуванням не передбачає прямого доступу контейнера до GPU. Тому для коректної взаємодії з графічними прискорювачами було розроблено кілька спеціалізованих інструментів, серед яких найбільш поширеними є NVIDIA Container Toolkit [13], nvidia-docker, NVIDIA Runtime, а також інструменти оркестрації, що

забезпечують GPU-менеджмент — такі як Kubernetes GPU Operator або Docker Compose із GPU-ресурсами.

NVIDIA Container Toolkit є основним інструментом, який забезпечує доступ контейнерів до GPU-ресурсів. Він побудований як сукупність модулів, що інтегрують Docker або інші контейнерні рушії з драйверами NVIDIA, встановленими на хості. Суть роботи Toolkit полягає у тому, що він надає контейнеру доступ до бібліотек CUDA, cuDNN, NCCL та інших системних компонентів, необхідних для роботи PyTorch та TensorFlow [13].

Його архітектура дозволяє контейнеру використовувати GPU без необхідності встановлювати драйвери всередині контейнера. Це значно спрощує побудову Docker-образів та робить їх легшими, оскільки контейнер містить лише бібліотеки користувацького рівня, тоді як драйвери залишаються на хості.

Першою реалізацією інтеграції GPU у Docker став проєкт nvidia-docker — проміжний шар між Docker Engine і NVIDIA драйверами. З появою NVIDIA Container Toolkit проєкт було інтегровано у Toolkit та переструктуровано.

Попри це, концептуально nvidia-docker залишив значний слід і нині продовжує функціонувати як компонент системи. Він дозволяє запускати Docker-контейнери з прямим доступом до GPU, використовуючи драйвери хост-системи та уникаючи потреби встановлення CUDA всередині контейнера.

Даний підхід зробив можливим стандартизований доступ до GPU у контейнерних середовищах, однак у сучасних системах використовується переважно як частина NVIDIA Runtime.

NVIDIA Runtime — це спеціалізований runtime-драйвер для Docker, який підключається через конфігураційний файл Docker Engine. Він забезпечує автоматичне встановлення бібліотек та прив'язку GPU до контейнера. Runtime відповідає за ін'єкцію відповідних бібліотек у контейнер під час його запуску, а також за коректну роботу CUDA у межах контейнера.

Цей компонент є обов'язковим для роботи ML-моделей, які виконуються на GPU, і дає можливість використання повного стеку CUDA у Docker-контейнерах.

У великих розподілених системах GPU-ресурси керуються за допомогою Kubernetes Device Plugin або NVIDIA GPU Operator. Останній забезпечує автоматичне встановлення драйверів, бібліотек та необхідних компонентів на вузлах кластера. У комплексних ML-інфраструктурах ці інструменти відіграють ключову роль, проте для невеликих або середніх систем їхня складність є надлишковою.

У деяких контейнерних середовищах використовується:

- Direct Host GPU Mapping — пряме підключення пристроїв до контейнера,
- OpenCL Runtime — альтернативний підхід для не-NVIDIA пристроїв,
- ROCm — платформа AMD для GPU-обчислень.

Однак у системах, орієнтованих на PyTorch і ResNet-архітектури, такі варіанти використовуються рідко через обмежену сумісність та складнішу конфігурацію.

Різні підходи до GPU-контейнеризації суттєво відрізняються за рівнем автоматизації, стабільності та сумісності з ML-фреймворками.

NVIDIA Container Toolkit забезпечує найбільш стабільний, підтримуваний і зрілий механізм взаємодії із GPU у Docker. Він повністю сумісний із PyTorch, TensorFlow та іншими бібліотеками, що робить його стандартом де-факто для ML-систем [13].

NVIDIA Runtime виступає низькорівневим компонентом, але без NVIDIA Container Toolkit функціонує неповноцінно.

Kubernetes GPU Operator є промисловим стандартом для великих кластерів, проте потребує складної інфраструктури, що виходить за рамки компактної системи, описаної у цій роботі.

OpenCL, ROCm та пряме мапування GPU застосовуються у специфічних випадках та не забезпечують тієї ж гнучкості, що і CUDA.

У промислових MLOps-рішеннях найчастіше використовується саме NVIDIA Container Toolkit як збалансоване, просте та стабільне рішення [2].

Іншим критичним аспектом у процесі тренування є керування даними. Тренування нейромережі зазвичай передбачає використання великих наборів даних, які неможливо зберігати у самому контейнері [8]. З цією метою у контейнеризації використовуються томи та директорії. Вони дозволяють зберігати моделі, журнали тренування та метадані поза контейнером. Завдяки цьому навіть після перезапуску або оновлення контейнера результати тренування залишаються доступними, що забезпечує безперервність процесу та можливість повторного аналізу.

У контейнеризованому середовищі також простіше управляти версіями моделей. Кожна нова модель, отримана у процесі тренування, зберігається у вигляді окремого артефакту, що дозволяє інтегрувати систему з CI/CD-конвеєрами. Автоматичне збереження моделей у заданому форматі спрощує контроль версій, а також надає можливість швидко повернутися до попередньої версії у разі помилки.

Важливо зазначити, що контейнеризоване тренування спрощує і сам процес масштабування. Якщо необхідно виконати одночасно кілька експериментів, це можна зробити шляхом запуску декількох екземплярів одного і того самого контейнера. Кожен контейнер працює незалежно від інших, що дозволяє розподілити експерименти між різними GPU або навіть різними фізичними серверами. Такий підхід неможливо реалізувати або він є значно складнішим у традиційних середовищах без контейнеризації.

Також контейнеризація сприяє інтеграції процесів тренування у загальну архітектуру системи. Контейнер з тренувальним модулем може включати у себе REST-інтерфейс, через який інші компоненти системи — наприклад, модуль моніторингу або веб-інтерфейс — можуть отримувати інформацію про поточний стан навчання, точність, втрати або час епохи. Таким чином, тренування перестає бути ізольованим процесом і стає частиною інтегрованої системи, у якій кожен компонент взаємодіє з іншими через стандартизовані API.

З точки зору експлуатації, тренування моделей у контейнерному форматі дозволяє уникнути проблеми залежності від конкретної операційної системи або її

конфігурації. Контейнер є переносним артефактом, який однаково працює як на Linux-сервері, так і у хмарному середовищі. Це дає змогу перенести тренування з локального середовища на хмарний сервер без додаткових налаштувань або ризику отримати некоректні результати через зміни у конфігурації системи.

Тому контейнеризація тренування нейромережових моделей забезпечує низку ключових переваг: від ізоляції середовища до можливості масштабування, від стабільності роботи GPU до гарантії відтворюваності результатів. У межах реалізованої системи такий підхід став невід’ємною частиною загальної архітектури, дозволивши інтегрувати процес тренування у CI/CD-конвеєр, забезпечити автономність роботи моделі та підтримувати безперервне оновлення сервісу інференсу. Завдяки цьому тренування у контейнеризованому середовищі перетворюється на керований, структурований і повторюваний процес, що повністю відповідає вимогам сучасних MLOps-практик.

2.3 Особливості впровадження DevOps і MLOps-підходів для AI-додатків

Розробка та експлуатація систем штучного інтелекту значно відрізняється від традиційного програмного забезпечення, що зумовлює необхідність застосування специфічних методологій і технічних підходів. Класичні DevOps-практики, такі як безперервна інтеграція, автоматизований деплоймент, інфраструктура як код та моніторинг, створили основу для побудови масштабованих та надійних програмних систем [3]. Однак у випадку робіт із нейромережевими моделями постає низка унікальних викликів, що потребують додаткових механізмів контролю життєвого циклу моделей, роботи з великими обсягами даних та забезпечення відтворюваності експериментів. Саме тому протягом останніх років DevOps-підходи були еволюційно розширені, утворивши окремий напрям — MLOps (Machine Learning Operations), який адаптує класичну розробку до потреб машинного навчання [2].

У традиційному DevOps центральним об’єктом роботи є вихідний код програмного забезпечення. Його зміни можуть бути легко відстежені, протестовані та автоматично інтегровані у CI/CD-конвеєр(рис. 4).

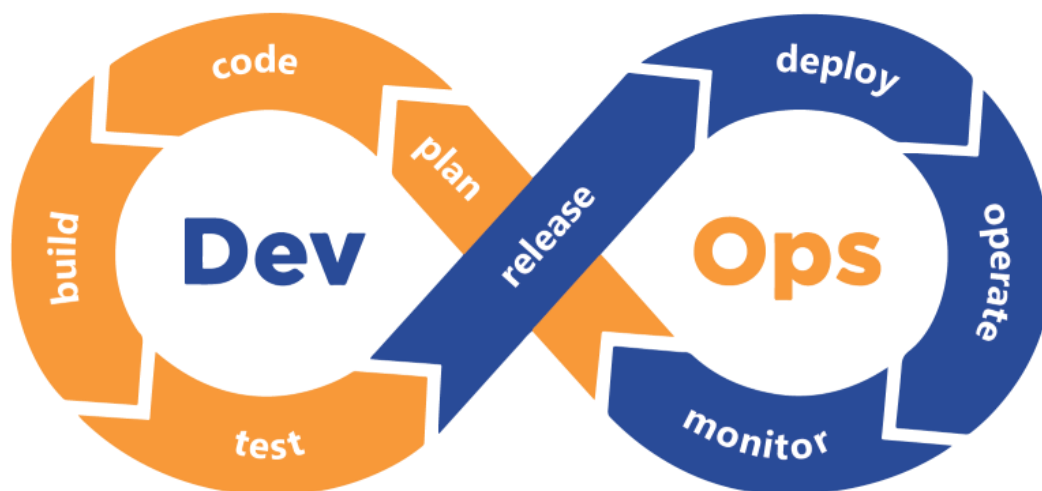


Рисунок 4 - Принцип роботи DevOps

Програмний артефакт є детермінованим: за незмінних умов компілятор завжди сформує ідентичний результат. У системах машинного навчання ситуація принципово інша. Модель залежить не лише від коду, а й від даних, випадкової ініціалізації, гіперпараметрів та апаратного середовища, у якому виконувалося тренування. Навіть незначна зміна структури даних може призвести до суттєво різних результатів моделі, що робить її нестабільним артефактом.

Ці особливості породжують кілька фундаментальних викликів: необхідність контролю версій не лише для коду, але й для моделей та навчальних датасетів; вимога забезпечення відтворюваності експериментів; потреба у постійному моніторингу якості моделей після розгортання; а також контроль дрейфу даних — поступових змін у характері вхідних даних, що можуть призвести до деградації моделі. Такі аспекти виходять далеко за межі класичного DevOps, що робить MLOps окремим напрямом із власною методологією та інструментами (рис. 5).

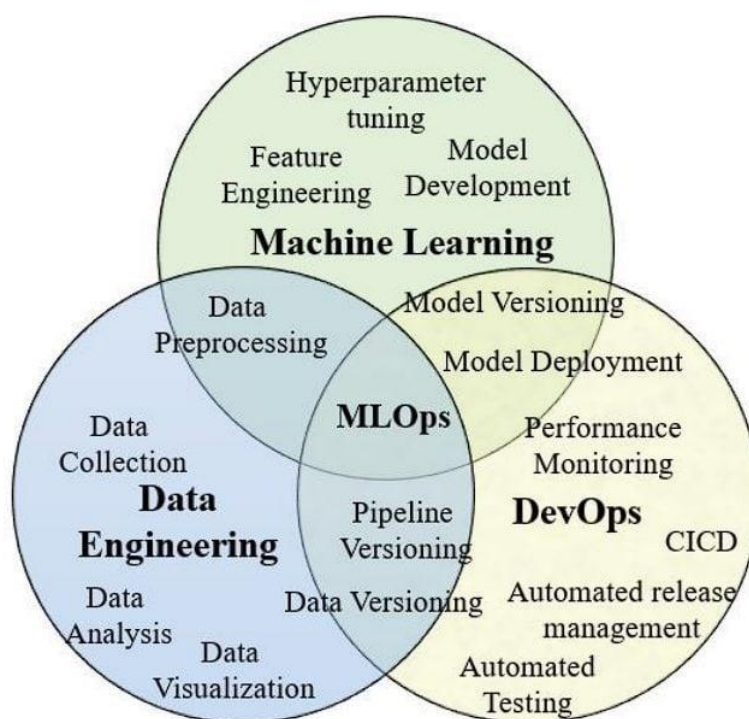


Рисунок 5 - Інструменти та методології, що формують MLOps

MLOps формує методологічну та технічну основу для побудови надійних, відтворюваних і масштабованих систем машинного навчання. На відміну від класичних DevOps-підходів, що орієнтуються на автоматизацію процесів розробки та поставки програмного забезпечення, MLOps розширює ці принципи з урахуванням специфіки роботи з даними, моделей та обчислювальних робочих навантажень. У сучасних AI-системах MLOps слугує критично важливим компонентом, оскільки модель ніколи не є статичним артефактом, а потребує постійного оновлення, контролю та перевірки [2].

У традиційних ML-процесах тренування моделі часто виконується вручну. MLOps перетворює цю задачу на керований, автоматизований процес. Це дозволяє автоматизувати запуск тренування, базуючись на подіях або зазначеному розкладу, оцінку моделі, створення артефактів та перемикання на нову модель.

Автоматизація забезпечує швидку реакцію на зміни та скорочує ризики людської помилки. Це дозволяє постійно вдосконалювати моделі без ручного втручання.

У DevOps контролюється лише код, натомість у MLOps потрібно керувати:

- версіями моделей,
- версіями датасетів,
- метаданими тренування,
- гіперпараметрами,
- результатами експериментів.

Модель змінюється не лише при модифікації коду, але й при зміні даних. Тому кожна модель має бути збережена з повним набором інформації, що дозволить відтворити її тренування. Цей принцип дозволяє проводити аудит, відкат, порівняння моделей та обирати найкращу версію з нетренованих.

Якщо у DevOps тестується лише код, то MLOps виконує тестування як функціональної валідації, так і валідації якості моделі, що включає точність, витрати, стабільність та відсутність деградації [2].

Перед розгортанням модель повинна пройти автоматизовані перевірки. Низька точність, нестача узагальнення або дрейф даних мають зупиняти деплоймент моделі. Таким чином, якість моделі стає критерієм її допуску до продакшену.

У класичних програмних системах CI/CD керує процесами побудови, тестування та деплойменту. У системах машинного навчання CI/CD розширюється: конвеєр повинен керувати також версіюванням моделей, запуском тренування, формуванням артефактів та оновленням інференс-сервісу. У реалізованій системі це виконано через GitHub Actions, де кожний пуш у основну гілку запускає побудову образів, оновлення сервісу та автоматичний blue–green deployment.

Моніторинг у MLOps суттєво відрізняється від DevOps. У DevOps моніторяться ресурси системи, стани сервісів та їх доступність. В AI-середовищі моніторинг доповнюється контролем якості моделі, швидкості інференсу та стабільності навчального процесу [2].

Модель, що працює в продакшені, повинна контролюватися постійно, адже її якість може змінюватися під впливом нових даних. Моніторинг дозволяє виявляти деградацію та автоматично запускати повторне тренування.

Основні принципи MLOps забезпечують здатність систем ШІ працювати як повноцінний виробничий продукт, а не як окремі експериментальні моделі. MLOps дозволяє:

- стандартизувати та автоматизувати життєвий цикл моделей,
- забезпечити стабільність і відтворюваність тренування,
- автоматизувати доставку моделей у продакшн,
- контролювати якість моделі протягом всього часу її використання,
- адаптувати AI-систему до змін у даних та середовищі.

Завдяки таким принципам AI-додатки стають надійними, масштабованими та готовими до реальної експлуатації у хмарних середовищах.

2.4 Способи виконання безперервного оновлення та масштабування моделей у хмарному середовищі

У сучасних цифрових продуктах системи штучного інтелекту перестали бути другорядними допоміжними модулями програмного забезпечення. Вони фактично стали ядром багатьох сервісів, визначаючи їхню якість, конкурентоспроможність і здатність адаптуватися до швидкозмінного ринку. Саме тому сучасні AI-рішення повинні забезпечувати не лише високу точність моделей чи швидкість інференсу, а й можливість їх безперервного оновлення й гнучкого масштабування у хмарних середовищах. У цьому контексті вимоги бізнесу перестають бути лише операційними; вони формують технічну архітектуру системи та визначають вибір інструментів і підходів.

Крім того, потреба у масштабуванні в ML-системах значно ширша: йдеться не лише про навантаження на сервер інференсу, але й про масштабування тренування моделей, роботу з GPU та розподіл обчислювальних ресурсів між кількома сервісами. У хмарному середовищі безперервність та масштабованість

стають вирішальними характеристиками, що забезпечують надійність, відмовостійкість та стабільність систем машинного навчання.

У сучасних компаніях системи штучного інтелекту дедалі частіше стають критично важливими елементами бізнес-процесів. Вони забезпечують персоналізацію, автоматизацію, аналітику та інтелектуальні функції, від яких залежить ефективність роботи всього підприємства. Саме тому стабільність та актуальність моделей машинного навчання перестають бути лише технічним питанням і переходять у площину бізнес-вимог. Безперервне оновлення та масштабованість моделей — фундаментальні характеристики, що визначають конкурентоспроможність продукту, рівень задоволеності користувачів та здатність системи адаптуватися до змін у реальному світі.

Насамперед бізнес очікує, що модель буде залишатися актуальною протягом усього циклу її використання. Дані у багатьох галузях змінюються надзвичайно швидко: користувацька поведінка еволюціонує, ринкові тенденції змінюються, з'являються нові сценарії використання, які модель раніше не бачила. Розрив між реальністю і тим, що модель під час тренування навчилася, призводить до деградації якості — а разом з нею і до прямих втрат бізнесу. Таким чином, виникає вимога до регулярного оновлення моделі або її повного перенавчання з урахуванням нових даних. Причому цей процес має бути повністю безперервним, щоб система не потребувала зупинки на час оновлення, адже навіть кілька хвилин недоступності сервісу можуть спричинити значні збитки, особливо для продуктів, що працюють у режимі реального часу.

Окремим, однак не менш важливим аспектом є вимога до гарантованої доступності сервісу інференсу. Користувачі сприймають інтелектуальні функції — класифікацію, рекомендації, обробку запитів — як базову функціональність, яка повинна працювати незалежно від внутрішніх технічних процесів. Компанія, що надає AI-сервіс, зобов'язана підтримувати рівень доступності, який часто визначається у межах 99.9% і вище. Це означає, що будь-яке оновлення моделі має виконуватися прозоро для кінцевого користувача. Модель може змінюватися, перевірятися, тестуватися або оновлюватися у фоновому режимі, але сервіс, який

її використовує, повинен залишатися доступним без будь-яких коливань у продуктивності.

Не менш важливою бізнес-вимогою є здатність системи швидко масштабуватися відповідно до змін у навантаженні. У реальному середовищі інтенсивність трафіку може бути дуже нерівномірною. У пікові години кількість запитів до сервісу інференсу може зростати у кілька разів, і система повинна реагувати на це автоматично, збільшуючи кількість екземплярів сервісу або розподіляючи навантаження між вузлами. У протилежних ситуаціях масштабування повинно виконуватися у зворотному напрямку, щоб уникати зайвих витрат на інфраструктуру. Бізнес хоче отримати модель, яка не лише швидко працює, а й економно використовує ресурси, адаптуючись до реальних потреб користувачів.

Критично важливим бізнесовим фактором є керованість і передбачуваність оновлень. Потрібно щоб нова версія моделі не тільки працювала коректно, але й не приводила до деградації сервісу. Будь-яке оновлення повинно проходити через формалізовані етапи перевірки, включно з тестуванням у реальних умовах, порівнянням з попередньою версією, аналізом її стабільності та виконанням health-check перевірок. Бізнес очікує, що система не дозволить випадкову або некоректну модель у продакшен — особливо в критичних сферах, таких як фінанси, медицина, захист інформації чи автономні системи.

У цьому контексті особливо важливою є можливість швидкого і передбачуваного відкату моделі до попередньої версії. Якщо нова модель виявиться менш точною, повільною або нестабільною, необхідно мати змогу повернутися до попереднього стану системи за секунди. Це знижує ризики, дозволяє уникнути довготривалих простоїв і забезпечує контроль над повним життєвим циклом моделі.

Ще однією важливою вимогою стає прозорість системи та її моніторинг. Необхідна можливість контролювати поведінку моделі не лише під час розгортання, але й після введення її в експлуатацію. Тут мова йде не про технічний моніторинг у вигляді CPU чи RAM, а про контроль якості моделі в

режимі реального часу: стабільність результатів інференсу, зміни в точності, наявність аномалій, відповідність очікуваному патерну поведінки. Система має надавати інженерам і менеджерам інструменти аналізу, що дозволяють своєчасно виявляти ознаки деградації моделі і запускати процес повторного тренування або оновлення.

Таким чином, бізнес-вимоги до безперервного оновлення та масштабування моделей формують комплексне бачення системи як гнучкого, самодостатнього й доступного середовища. Це не просто набір технічних характеристик, а фундаментальні умови, без яких сучасний AI-продукт не може забезпечити стабільну роботу, адаптуватися до змін даних або відповідати очікуванням користувачів. Ці вимоги визначають архітектурні рішення, вибір технологічних компонентів і загальний підхід до побудови системи машинного навчання у хмарі — саме вони лежать в основі розробленої у цій роботі системи автоматизованого оновлення та масштабування ШІ-додатків.

Безперервне оновлення моделей у системах штучного інтелекту є однією з найбільш важливих та водночас складних складових сучасної MLOps-архітектури. На відміну від звичайних веб-додатків, де оновлення переважно стосуються програмного коду, у AI-системах необхідно розгортати нові версії моделей, які можуть відрізнятися за точністю, швидкістю роботи, структурою чи навіть вимогами до ресурсів. Цей процес має відбуватися без будь-якого негативного впливу на доступність сервісу, оскільки для більшості бізнесових застосувань штучний інтелект виконує ключові функції, а навіть короткочасна недоступність може спричинити значні операційні втрати. Саме тому у галузі сформувалося кілька підходів, які дозволяють забезпечити плавний, передбачуваний і контрольований перехід від однієї версії моделі до іншої, без зупинки інференс-сервісу та з мінімальними ризиками.

Одним із найбільш відомих і поширених підходів є blue–green deployment. Його сутність полягає у створенні двох паралельних середовищ: активного (blue) і нового (green). Нова модель, разом із сервісом, що виконує інференс, розгортається в новому середовищі повністю ізольовано від поточного трафіку.

Це дозволяє проводити повний цикл тестування — від перевірки працездатності API до валідації моделі на еталонних даних — без ризику для користувачів. Лише після того, як система переконалася у коректності роботи оновленої версії, трафік перемикається з blue на green. Такий підхід фактично усуває будь-які простоти, дозволяє проводити оновлення у будь-який час та забезпечує миттєву можливість відкату, якщо нова версія проявить аномалії у поведінці. Саме тому blue–green deployment вважається одним із найбільш надійних методів оновлення моделей, і він широко використовується в індустрії, зокрема у виробничих середовищах з високими вимогами до доступності [1].

Іншим важливим підходом є canary deployment, який більш тонко контролює процес розгортання. Замість повного переключення трафіку одразу, нова модель отримує лише невелику частину запитів. Це дозволяє оцінити її поведінку на реальних даних і справжніх користувацьких сценаріях. Якщо нова модель демонструє стабільність, її частка у трафіку поступово збільшується, доки вона не стане основною. Якщо ж виникають проблеми — зниження точності, збільшення часу відповіді або непередбачувані помилки — система автоматично повертається до попередньої версії. Цей метод особливо корисний у великих системах із мільйонними обсягами запитів, де можливість дрібного поступового переходу знижує ризики та дозволяє приймати технічно виважені рішення на основі живих даних.

З-поміж інших стратегій значне місце посідає shadow deployment. У цьому підході нова модель працює паралельно зі старою, але її результати не впливають на відповіді, що отримує користувач. Система дублює вхідні запити і надсилає їх обом моделям, після чого порівнює їхні відповіді. Це дає можливість оцінити якість нової моделі максимально наближено до реальних умов, без будь-якого ризику для користувацького досвіду. Shadow deployment особливо корисний, коли оновлення моделі є складним або архітектурно значущим, і потрібно переконатися, що нова версія працює коректно у всіх можливих сценаріях використання.

У великих інфраструктурах, особливо побудованих на Kubernetes, широкого застосування набуває підхід rolling update. У цьому випадку нова версія моделі поступово замінює стару на рівні окремих екземплярів сервісу. Кластери, що підтримують балансування навантаження, дозволяють оновлювати моделі поетапно, не впливаючи на загальну доступність. Проте цей метод добре працює лише у масштабованих інфраструктурах, де є кілька екземплярів сервісу інференсу, і де кожен окремий вузол може бути тимчасово оновлений без втрати загальної продуктивності.

Кожен із перелічених підходів вирішує власний клас задач і має як переваги, так і обмеження. Blue–green deployment забезпечує найпростішу модель відкату і найвищу стабільність, але вимагає більше ресурсів [1]. Canary deployment є більш гнучким, проте потребує складнішої оцінки якості моделей у режимі реального часу. Shadow deployment дозволяє виконувати тестування у реальних умовах, однак не підходить для оцінки продуктивності під навантаженням. Rolling update ефективний у масштабованих середовищах, але важко реалізується в мінімалістичних інфраструктурах.

Важливо зазначити, що незалежно від обраного підходу процес безперервного оновлення завжди включає кілька критичних елементів: автоматизоване побудування та пакування моделі, перевірку її працездатності, тестування у контрольованому середовищі або на реальному трафіку, а також механізм миттєвого відкату. У поєднанні ці елементи формують надійну архітектурну основу, що дозволяє системам штучного інтелекту залишатися актуальними та стабільними, навіть коли моделі оновлюються регулярно та часто.

Таким чином, підходи до безперервного оновлення моделей не лише різноманітні, але й глибоко інтегровані у загальний MLOps-процес. Вони забезпечують адаптивність системи, її готовність до змін у даних і можливість швидко реагувати на нові виклики без втрати стабільності та доступності. Реалізація таких підходів є однією з фундаментальних ознак зрілої, виробничої інфраструктури штучного інтелекту.

Масштабування систем штучного інтелекту у хмарному середовищі є одним із ключових аспектів, що визначає їхню продуктивність, відмовостійкість та здатність задовольняти зростаючі обсяги користувацьких запитів. На відміну від традиційних програмних додатків, де збільшення навантаження здебільшого впливає лише на обчислювальні ресурси сервера, у випадку моделей машинного навчання масштабування вимагає більш комплексного підходу, оскільки воно зачіпає не лише сервіс інференсу, але й саму модель, механізми тренування, обробку даних, роботу з GPU та інфраструктуру для підтримки оновлень. У таких системах масштабування стає невід'ємною частиною загальної архітектури, що дозволяє забезпечити високу продуктивність, плавне обслуговування запитів та можливість паралельної обробки великих потоків даних.

У сучасних AI-рішеннях масштабування зазвичай розглядають у двох вимірах: масштабування сервісу інференсу та масштабування процесу тренування моделі. Хоча обидва процеси взаємопов'язані, вони вирішують різні задачі. Масштабування інференсу спрямоване на збільшення кількості запитів, які система може обробляти за одиницю часу, а також на підвищення доступності сервісу. Якщо кількість користувачів або інтенсивність їх взаємодії з системою зростає, сервіс повинен мати змогу миттєво збільшити свою пропускну здатність без зниження якості або швидкодії. Це особливо важливо для реального бізнесу, де часові затримки можуть призвести до втрат — наприклад, у фінансових системах, онлайн-магазинах або медичних сервісах, де AI-рішення використовуються безпосередньо у критичних процесах.

Масштабування інференсу може відбуватися як вертикально, так і горизонтально. Вертикальне масштабування передбачає збільшення ресурсів одного вузла — наприклад, перехід на GPU замість CPU або використання серверів із більшою кількістю ядер, оперативної пам'яті або підтримкою прискореної обробки даних. Такий підхід дозволяє швидко збільшити продуктивність, але його межі визначаються апаратними можливостями. Горизонтальне масштабування, натомість, передбачає розгортання кількох ідентичних екземплярів сервісу інференсу. Це дозволяє розподіляти навантаження

між кількома контейнерами або вузлами, що значно підвищує відмовостійкість — у разі збою одного екземпляра інші продовжують роботу. Горизонтальне масштабування також полегшує обслуговування оновлень, оскільки окремі вузли можуть оновлюватися поетапно, без впливу на загальну доступність системи.

Особливістю масштабування інференсу в AI-системах є те, що моделі можуть значно відрізнятися за вимогами до ресурсів. Наприклад, моделі комп'ютерного зору часто потребують GPU навіть для інференсу, тоді як простіші моделі можуть оброблятися на CPU. Це вимагає інтелектуального планування ресурсів, а також здатності системи гнучко перемикатися між різними конфігураціями залежно від навантаження. У хмарному середовищі такі механізми можуть бути реалізовані через спеціалізовані оркестратори або за допомогою контейнерних систем, які враховують доступність GPU, оптимізують маршрутизацію запитів та дозволяють ефективно розподіляти обчислення між кількома вузлами.

Масштабування тренування моделі, у свою чергу, є більш спеціалізованим процесом. Виробничі моделі можуть містити мільйони або навіть мільярди параметрів, і їх тренування потребує колосальних обчислювальних ресурсів. Тут масштабування може відбуватися за рахунок використання кількох GPU або цілих кластерів обчислювальних вузлів. У бібліотеках PyTorch або TensorFlow існують спеціальні інструменти для розподіленого тренування [16], що дозволяють виконувати паралельні обчислення, синхронізувати ваги між вузлами та забезпечувати узгодженість моделі [8]. У хмарних системах це часто комбінується з керуванням чергами задач, автоматичним плануванням тренування або обчислювальним autoscaling, коли система самостійно визначає оптимальну кількість ресурсів для навчання.

У системах, де інференс і тренування використовуються одночасно, важливою стає можливість ізоляції навантаження. Незалежно від потужності інфраструктури, процес тренування не повинен впливати на стабільність інференсу. Це означає, що системи часто розділяють робоче навантаження між різними контейнерами, вузлами або навіть окремими GPU. Такий підхід дозволяє

одночасно тренувати нову версію моделі й обслуговувати користувацькі запити, що робить можливим безперервне оновлення моделі без переривання сервісу.

У хмарних середовищах масштабування також тісно пов'язане з економічною ефективністю. Забезпечення високої доступності не повинно перетворюватися на надмірне споживання ресурсів, адже обчислення на GPU або розподілених кластерах можуть бути дорогими. Саме тому сучасні AI-системи активно використовують механізми автоматичного масштабування, які дозволяють збільшувати кількість інстансів сервісу під час пікових навантажень та зменшувати їх у періоди спаду. Це гарантує оптимальне співвідношення між продуктивністю та вартістю, що є критично важливим для довготривалої підтримки AI-продукту.

Загалом масштабування моделей і сервісів інференсу становить невід'ємну частину зрілої MLOps-практики [18]. Воно забезпечує не лише стабільність і швидкодію системи, але й дозволяє витримувати зростання користувацької бази, реагувати на зміни в поведінці користувачів, швидко адаптуватися до нових вимог і підтримувати актуальність моделі протягом тривалого часу. Саме завдяки масштабуванню штучний інтелект здатний поєднувати складні моделі з високими вимогами до ресурсів із реальними бізнес-процесами, що потребують стабільності і безперервної роботи. Це робить масштабування ключовим елементом сучасної архітектури AI-додатків у хмарних середовищах.

Контейнеризація стала одним із ключових технологічних проривів у сучасній інфраструктурі розгортання програмних систем, а у сфері штучного інтелекту її значення ще більше зросло завдяки вимогам до стабільності середовища, відтворюваності тренувань та ізольованості моделей. Коли йдеться про безперервне оновлення моделей, контейнеризація виконує не просто роль зручного інструмента — вона фактично стає фундаментальною основою всієї архітектури оновлень, визначаючи можливість швидкого розгортання нових версій моделі, безпечного тестування та миттєвого перемикання між різними конфігураціями сервісів.

Головною перевагою контейнеризації є повна ізоляція середовищ. Моделі машинного навчання є надзвичайно чутливими до змін у залежностях, версіях бібліотек, драйверів GPU, конфігурацій CUDA та інших компонентів. Навіть незначні розбіжності у версіях програмних пакетів можуть призвести до некоректної роботи або відмінності результатів інференсу. Традиційні підходи до розгортання у таких умовах часто потребували складного налаштування, що робило процес оновлення ризикованим. Контейнери вирішують цю проблему методом упаковки повного виконуваного середовища разом із моделлю і кодом. Це означає, що нова версія моделі розгортається у точно такому ж програмному оточенні, в якому вона була натренована та протестована, що практично усуває ризики несумісності.

У контексті безперервного оновлення контейнеризація дає змогу працювати з кількома версіями моделі одночасно. Така можливість є критично важливою у сценаріях blue–green або canary deployment, де нове середовище повинно існувати паралельно зі старим, щоб пройти повний спектр перевірок. Контейнери дозволяють розгорнути новий екземпляр сервісу за лічені секунди, не порушуючи роботу вже активної інфраструктури. Завдяки визначеному контейнерному образу інженер точно знає, що green-версія сервісу ідентична тій, що була побудована у CI/CD-конвеєрі, включно з бібліотеками, залежностями та конфігураційними файлами.

Контейнеризація також спрощує механізми відкату до попередньої версії моделі. У традиційних підходах повернення до стабільної версії вимагало складних операцій із заміни файлів, перевірки середовища або навіть повного перезапуску серверів. Контейнерний підхід дозволяє миттєво перейти до попереднього образу, адже кожна версія сервісу є окремим контейнером або набором контейнерів, які можуть повторно запускатися у будь-який момент. Таким чином, rollback фокусує увагу не на перескладанні або перевстановленні компонентів, а просто на перемиканні трафіку між різними контейнерними інстансами.

Не менш важливим аспектом контейнеризації є її тісна інтеграція з інструментами автоматизації та CI/CD. Оскільки контейнер є самодостатнім артефактом, система CI/CD може легко створити новий образ моделі, протестувати його і передати до середовища виконання. Це забезпечує те, що кожен етап оновлення — від тренування до розгортання — має свій контрольований артефакт, який може бути перевірений, задокументований або відтворений. У виробничих ML-рішеннях це особливо важливо, адже моделі потребують не лише перевірки коду, але й контролю метрик, версій та відповідності специфікаціям.

Крім того, контейнеризація надає можливість легко масштабувати сервіси інференсу, що є важливим доповненням до безперервного оновлення. Уся логіка масштабування прив'язана не до окремих програмних компонентів, а до контейнерів, які можуть запускатися у кількох екземплярах відповідно до потреб системи. Завдяки цьому нова версія моделі може бути розгорнута на обмеженій кількості інстансів для тестування (як у canary-підході), а пізніше — розширена на весь кластер. Контейнеризація робить масштабування незалежним від програмних деталей, що суттєво зменшує складність системи та підвищує її надійність.

Важливу роль відіграє і те, що контейнеризація чудово підходить для роботи з GPU. Система, яка підтримує роботу з GPU всередині контейнерів, отримує додаткове стратегічне значення: по-перше, вона забезпечує передбачуваність робочого середовища, а по-друге — спрощує процес повторного тренування і тестування моделей перед введенням у продакшн.

Таким чином, роль контейнеризації у безперервному оновленні моделей виходить значно за межі традиційного розуміння. Це не просто спосіб упаковки програм і моделей, а потужний архітектурний механізм, що забезпечує стабільність середовищ, можливість ізолюваного тестування, гнучкість оновлень, швидке масштабування і передбачуваність виробничих процесів. Контейнеризація стає тим технічним каркасом, на якому вибудовується повноцінний MLOps-цикл: від тренування моделі до її безперервного оновлення в хмарному середовищі.

Саме завдяки їй системи штучного інтелекту можуть еволюціонувати швидко, безпечно та з мінімальними ризиками для кінцевих користувачів [18].

Перевірка працездатності моделі є однією з ключових складових процесу безперервного оновлення в AI-системах, оскільки саме на цьому етапі визначається, чи є нова версія моделі достатньо стабільною, точною та сумісною з виробничим середовищем, щоб бути впровадженою без ризику для сервісу. На відміну від класичного тестування програмного забезпечення, де перевіряється логіка виконання коду, тестування моделей машинного навчання включає комплексну оцінку, що охоплює як технічні аспекти (завантаженість, коректність API, використання ресурсів), так і якісні (точність, стабільність, відсутність деградації). Саме тому механізми валідації моделі перед розгортанням формують окремий пласт у MLOps-процесах і потребують значно більшої уваги, ніж звичайне тестування компонентів [18].

Після завершення тренування модель повинна пройти так звану структурну перевірку, яка підтверджує, що файл моделі є коректним, має правильний формат і може бути завантажений без помилок у середовищі інференсу. У цьому контексті важливо переконатися, що модель не була пошкоджена під час збереження, що її ваги мають очікувану форму, а сама архітектура відповідає версії коду інференсу. Це особливо критично у виробничих системах, де навіть невелика несумісність між тренувальним та інференс-кодом може повністю зупинити сервіс. Перевірка завантаженості у контейнері тестового середовища дає змогу гарантовано переконатися у працездатності моделі до того, як вона буде використана для обробки запитів користувачів.

Другим важливим аспектом є функціональна перевірка, яка підтверджує, що оновлена модель поводить себе коректно з точки зору API й логіки інференсу. Йдеться не про оцінку точності моделі, а про перевірку того, що інференс-сервіс правильно приймає та обробляє запити, не спричиняє внутрішніх помилок і видає відповіді у потрібному форматі. У більшості систем такий тест реалізується через окремі health-check ендпоїнти, які дозволяють автоматизовано перевірити стан сервісу.

Невід'ємною частиною механізмів перевірки є тестування моделі на контрольному наборі даних, який виконується перед її активацією. Такий підхід дозволяє переконатися, що модель не втратила точності внаслідок тренування, що її метрики відповідають очікуваним значенням і що вона демонструє стабільну поведінку на репрезентативній вибірці. На практиці цей процес може включати як базові sanity-check тести (наприклад, перевірку, що модель розпізнає декілька ключових зразків), так і детальніші метрики точності, втрат, F1-score чи інших показників залежно від типу задачі. У виробничих системах важливо, щоб ці тести були максимально легкими й не займали багато часу, адже затримка на цьому етапі може уповільнити весь конвеєр CI/CD.

Ще одним критичним механізмом оцінки працездатності є перевірка моделі в умовах реального навантаження, навіть якщо це відбувається без доступу користувача до результатів. У таких сценаріях модель проганяється через shadow deployment, коли вона отримує дублікати реальних запитів і генерує відповіді, які порівнюються з результатами активної моделі. Це дозволяє отримати повну картину її поведінки у реальному середовищі, включно з затримками, використанням ресурсів, стабільністю та коректністю вихідних даних. Shadow-режим зазвичай застосовується для оновлення критичних або високоризикових моделей, але навіть у легких системах він дає змогу отримати додаткову впевненість перед активацією нової версії.

Окремо слід відзначити важливість аналізу продуктивності моделі, оскільки навіть невелике збільшення часу інференсу може суттєво вплинути на загальну швидкість системи при високому навантаженні. Виробничі системи з поточними або інтенсивними запитами особливо чутливі до таких змін. Саме тому перед активацією нової моделі необхідно виміряти її час відповіді, використання процесорних і графічних ресурсів та стабільність у довготривалій роботі. У складних ML-рішеннях оцінюється також рівень коливань продуктивності, оскільки нестабільні моделі можуть у певних ситуаціях різко збільшувати затримки або навіть спричиняти аварійні ситуації.

Вагомою складовою перевірки працездатності є можливість фіксації метаданих, збережених під час тренування. У системах, де фіксуються метрики тренувального процесу, епохи, гіперпараметри та інші характеристики, інженери отримують змогу переконатися, що модель тренувалася відповідно до очікуваних параметрів, не демонструвала дивних аномалій і завершилася коректно. Це також дозволяє виявити порушення, які можуть бути непомітними під час звичайної перевірки інференсу, але свідчать про помилки у процесі навчання.

Таким чином, механізми перевірки працездатності моделі становлять комплексний процес, який включає оцінку завантаженості, функціональної правильності, точності, стабільності, продуктивності та відповідності виробничим стандартам. Це не одномоментна дія, а багаторівнева система захисту, що гарантує безпечну інтеграцію нових моделей без ризику для продукту чи користувачів. Саме завдяки такому підходу системи штучного інтелекту стають надійними та передбачуваними навіть у середовищах, де моделі оновлюються часто, автоматично та безперервно.

Організація централізованого зберігання моделей є одним із ключових компонентів сучасної MLOps-інфраструктури, оскільки саме від способу збереження залежить можливість відтворення експериментів, контроль версій, швидкий доступ до артефактів, а також стабільність процесів безперервного оновлення у виробничому середовищі [18]. Модель у контексті машинного навчання — це не просто файл з вагами; це критичний елемент системи, який може мати різні версії, конфігурації, залежності, метрики та відповідні набори даних. Саме тому питання її зберігання виходить далеко за межі простого створення каталогу і перетворюється на складову стратегічної архітектури, що забезпечує надійність та керованість AI-системи в цілому.

У виробничих системах зберігання моделей повинне гарантувати повну відтворюваність процесу тренування. Це означає, що кожна модель має бути збережена разом з метаданими, які описують умови її створення: версію навчальних даних, гіперпараметри, конфігурацію середовища, тривалість тренування, використовувані апаратні ресурси, досягнуті метрики тощо. У

практичному сенсі це створює можливість повернутися до будь-якої попередньої версії, порівнювати результати різних експериментів, проводити аудит процесу навчання або відтворювати тренування для забезпечення стабільності моделі. У системах без якісної організації зберігання моделей ці можливості суттєво обмежені, що призводить до втрати контролю над життєвим циклом моделі та до потенційних збоїв при оновленні.

Також важливим аспектом є питання структурованості зберігання. Централізоване сховище повинно мати чітку організацію, що дозволяє швидко знаходити потрібну версію моделі. У найпростішому випадку це може бути каталог із моделями, що позначаються часовими мітками або номерами версій. Але у складних системах, де моделі тренуються регулярно й автоматично, така організація має бути доповнена засобами метаданих, які дозволяють автоматично вибирати найкращу модель або готуватися до деплойменту. Важливо, щоб кожна модель була самодостатньою — щоб її завантаження не залежало від зовнішніх випадкових факторів або змін у середовищі. Завдяки цьому модель може бути швидко розгорнута у новому контейнері або використана у процесі відкату без ризику несумісності.

Одним із ключових завдань централізованого зберігання є забезпечення безперервної взаємодії між сервісами тренування та інференсу. У системах, де оновлення моделі відбувається автоматично, сервіс тренування повинен мати можливість зберегти нову модель у спільному сховищі, а сервіс інференсу — отримати доступ до неї незалежно від власного стану. Це унеможливорює ситуації, коли модель зберігається локально в тренувальному контейнері або тимчасово недоступна сервісу інференсу. З цієї причини в системах MLOps широко використовуються персистентні томи — механізм, який дозволяє зберігати моделі поза межами контейнерів, забезпечуючи їх доступність під час оновлень, перезапуску або масштабування сервісів.

Централізоване зберігання моделей також виконує важливу роль у забезпеченні безперервного оновлення. Під час blue–green deployment або інших методів безперервного розгортання, де нова версія моделі повинна бути

заздалегідь доступною у сховищі для тестового середовища. Якщо модель не буде доступною в момент тестування сервісу, деплоймент буде неможливим або нестабільним. Тому централізоване сховище фактично перетворюється на джерело інформації для всієї системи, де кожна модель зберігається у формі артефакту, готового до тестування і використання. Це дозволяє автоматизувати процеси, виключивши необхідність ручного втручання при оновленні моделей.

Окремої уваги заслуговує питання безпеки та контролю доступу. Моделі можуть містити як конфіденційну, так і комерційно цінну інформацію. Тому централізоване сховище має підтримувати контроль доступу, журнали змін та механізми аутентифікації. Це дозволяє запобігти несанкціонованим модифікаціям моделей та забезпечує історичний запис про те, хто і коли виконував тренування або розгортання. У великих організаціях такі механізми є стандартом, а у хмарних середовищах вони часто реалізуються на рівні служби збереження, що дає можливість забезпечити цілісність та захист моделей під час оновлень.

Таким чином, організація централізованого зберігання моделей є базовим елементом, який забезпечує стабільність, відтворюваність та керованість системи штучного інтелекту. Вона створює основу для автоматизації безперервного оновлення, полегшує інтеграцію між сервісами, підвищує безпеку, забезпечує аудит та дозволяє системі масштабуватися без втрати якості чи стійкості. Без правильно організованого сховища моделей будь-який MLOps-процес залишається вразливим та непередбачуваним, тоді як грамотно побудована модельна база перетворює систему на керований, прогнозований та гнучкий механізм, здатний до постійного розвитку [18].

Висновки до розділу

У ході цього розділу було здійснено комплексне дослідження архітектурних рішень, технологічних інструментів та підходів, що визначають ефективність побудови системи автоматичного розгортання ШІ-застосунків у хмарному середовищі. У цьому аналізі було приділено увагу як фундаментальним технологіям контейнеризації та оркестрації, так і спеціалізованим методологіям

DevOps і MLOps, які забезпечують повний цикл життєдіяльності моделей машинного навчання — від тренування до безперервного оновлення та експлуатації.

Дослідження можливостей Docker, Docker Compose, Kubernetes, Docker Swarm та Nomad дозволило сформувати цілісне бачення сучасних підходів до побудови інфраструктури для ІІІ-систем. Було визначено, що саме контейнеризація виступає основою надійного розгортання і масштабування, забезпечуючи стабільність середовища, ізоляцію залежностей, відтворюваність експериментів та гнучкість архітектури. Серед можливих інструментів оркестрації було проаналізовано їхні переваги та недоліки, що дало змогу сформувати обґрунтований вибір на користь Docker Compose як оптимального рішення для системи, масштаби якої не потребують повноцінного Kubernetes-кластеру, але водночас вимагають зрозумілого, передбачуваного і стабільного механізму керування кількома сервісами.

У межах підрозділу, присвяченого тренуванню нейромережових моделей у контейнеризованому середовищі, було звернуто увагу на вибір бібліотек машинного навчання та інструментів для використання GPU. Порівняння PyTorch і TensorFlow продемонструвало переваги PyTorch у контексті прототипування, динамічних обчислень і гнучкості побудови моделей, що робить цю бібліотеку більш придатною для досліджуваної системи. Аналіз інструментів роботи з графічними прискорювачами показав важливість NVIDIA Container Toolkit як універсального рішення, що забезпечує коректну інтеграцію GPU у середовище Docker та дозволяє виконувати навчання та інференс у стабільних, контрольованих умовах.

Розгляд DevOps і MLOps-підходів дав змогу усвідомити, наскільки важливими вони є для створення виробничих AI-систем. Було виявлено, що класичні DevOps-практики не можуть повністю задовольнити потреби моделей машинного навчання, де актуальність, якість та стабільність моделі змінюються залежно від даних і параметрів тренування. Тому впровадження MLOps стало логічним продовженням архітектурного проектування, забезпечивши

автоматизацію тренування, контроль версій моделей, безперервне оновлення, моніторинг та інтеграцію моделей у CI/CD-конвеєр. У результаті було сформовано цілісне бачення того, як злагоджені процеси DevOps і MLOps можуть забезпечити надійність і масштабованість системи автоматичного розгортання ІІІ-додатків.

Окрема увага у розділі була приділена способам безперервного оновлення та масштабування моделей. Розглянуто такі підходи, як blue–green deployment, canary deployment, shadow deployment та rolling update, що дозволяє оцінити їхні переваги й можливості у контексті промислового застосування. Було встановлено, що для створюваної системи найбільш збалансованим і надійним механізмом є blue–green deployment, який гарантує повне відокремлення нової версії моделі від активного середовища, забезпечує її перевірку без ризику для користувачів та дозволяє виконувати миттєвий відкат у разі потреби. Також було проаналізовано питання масштабування, де контейнеризація у поєднанні з можливостями Docker Compose забезпечує достатню гнучкість як для інференсу, так і для тренування моделей, дозволяючи системі реагувати на зміни у навантаженні.

У підсумку проведеного дослідження було вибрано цілком обґрунтований технологічний стек та архітектурні рішення для реалізації системи автоматичного розгортання ІІІ-застосунків у хмарному середовищі. До них належать Docker та Docker Compose як основа контейнеризації й оркестрації, PyTorch як базова бібліотека машинного навчання, NVIDIA Container Toolkit для забезпечення GPU-підтримки, а також blue–green deployment як основний механізм безперервного оновлення. Інфраструктура на основі цих компонентів дозволяє досягти високої стабільності, масштабованості та автоматизації процесів, що є необхідними для ефективного функціонування сучасних систем штучного інтелекту.

Таким чином, другий розділ сформував теоретичне та технологічне підґрунтя, яке визначає архітектуру розробленої системи. Отримані результати заклали основу для практичної реалізації розроблюваної системи.

3 ОПИС РЕАЛІЗОВАНОЇ ХМАРНО-ОРІЄНТОВАНОЇ СИСТЕМИ ТА ЇЇ КОМПОНЕНТІВ

3.1 Характеристика системи та основних компонентів

Розроблена хмарно-орієнтована система автоматизації розгортання ІІІ-застосунків являє собою завершений комплекс із взаємопов'язаних компонентів, які забезпечують повний життєвий цикл ІІІ-моделей — від етапу тренування до їх безперервного оновлення та інтеграції у виробниче середовище. Архітектура системи побудована за принципами хмарних підходів та використовує контейнеризацію, що забезпечує модульність, тактовність оновлень, передбачуваність оточення та можливість розгортання у будь-яких сучасних хмарних платформах.

Основу системи становить набір мікросервісів, розгорнутих у контейнерах Docker, кожен із яких виконує чітко визначені функції та взаємодіє з іншими компонентами через стандартизовані REST API. Усі сервіси працюють у межах спільної віртуальної мережі Docker, використовують персистентні томи для зберігання моделей та логів і можуть масштабуватися відповідно до потреб середовища.

Архітектура також передбачає використання Load Balancer, який відповідає за маршрутизацію зовнішніх запитів. Це дозволяє розділяти трафік між екземплярами системи, забезпечувати доступність сервісів та, у разі використання декількох інстансів ai_frontend чи ai_api, гарантувати їх рівномірне навантаження. У контексті механізму оновлення Load Balancer також виконує роль точки перемикання між двома версіями сервісу інференсу, що дає змогу впроваджувати моделі без зупинки системи та без ризику для користувача.

Центральним компонентом системи є сервіс інференсу ai_api, який відповідає за виконання класифікації зображень, перевірку стану моделі, її перезавантаження та обробку основних виробничих запитів. У системі одночасно можуть існувати дві його версії — blue та green, що забезпечує механізм безперервного розгортання. Активна версія отримує запити користувачів, тоді як альтернативна версія використовується для попереднього тестування оновленої

моделі. Така структура робить можливим перехід між версіями без простою, при цьому гарантуючи, що оновлена модель є працездатною та пройшла всі перевірки до моменту активації.

Другим ключовим модулем є сервіс тренування `ai_trainer`, який відповідає за формування нових версій нейронної моделі. Він виконує тренування мережі ResNet-18 на основі датасету CIFAR-10, підтримує роботу на GPU, зберігає модель у спільному томі `./models/` та формує метадані тренування у каталозі `./trainer_logs/`. Після завершення процесу тренування сервіс автоматично ініціює оновлення моделі в інференс-сервісі, надсилаючи запит на ендпоінт `/reload`, що забезпечує безперервність MLOps-процесу.

Для забезпечення стабільності, відмовостійкості та спостережуваності системи впроваджено окремий модуль моніторингу `ai_monitor`. Він періодично перевіряє стан сервісів `ai_api` та `ai_trainer`, збирає системні метрики, аналізує використання ресурсів (CPU, RAM, Disk), а також формує історичні журнали у вигляді JSON-файлу [9]. Модуль моніторингу є центральним джерелом інформації про технічний стан системи та використовується як ендпоїнтами CI/CD, так і графічним інтерфейсом.

Останнім важливим компонентом є веб-інтерфейс `ai_frontend`, який дозволяє користувачу взаємодіяти з системою: запускати тренування, надсилати зображення на класифікацію, переглядати стан сервісів, аналізувати метрики моніторингу та контролювати оновлення моделей. `ai_frontend` виступає центральною точкою взаємодії між користувачем і всіма сервісами, виконуючи роль інтеграційного інструмента та адміністративної панелі управління системою.

У межах системи також працює CI/CD-конвеєр, який виконує автоматичну перевірку змін у репозиторії, побудову контейнерних образів, ініціює розгортання нової версії сервісів та керує процесом blue-green оновлення. Його робота безпосередньо пов'язана із запуском спеціального скрипта `deploy.sh`, що визначає активну версію сервісу інференсу, піднімає альтернативний стек, виконує health-check нової версії та за необхідності переключає трафік через Load Balancer.

На рисунку 6 наведено структурну схему реалізованої системи, яка відображає взаємозв'язки між її основними компонентами, маршрутизацію запитів, взаємодію при тренуванні та оновленні моделі, а також механізм перемикавання між blue та green серверами.

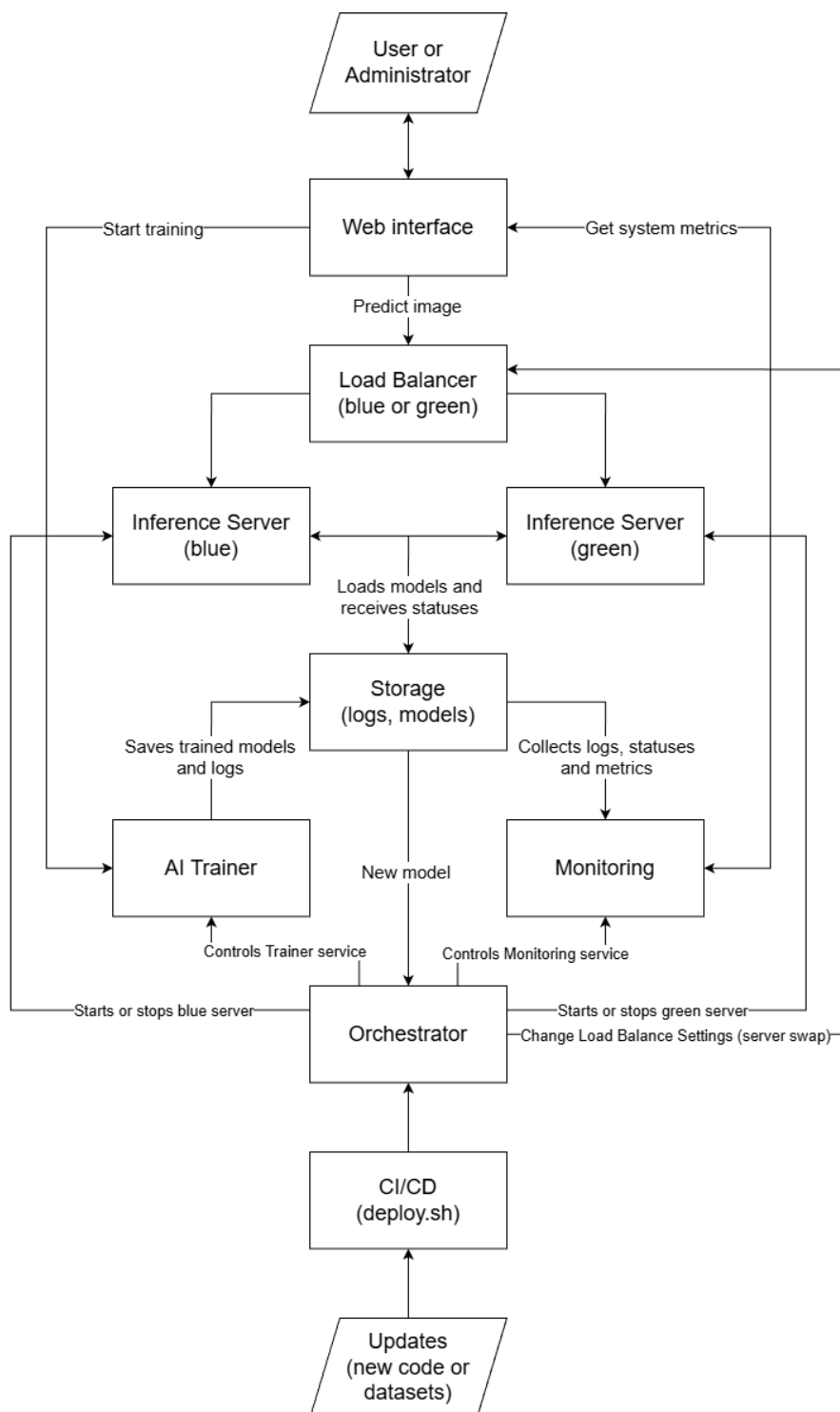


Рисунок 6 - Архітектура реалізованої хмарно-орієнтованої системи автоматизації розгортання ШІ-застосунків

Таким чином, система представляє собою модульну, масштабовану та повністю автоматизовану інфраструктуру, здатну забезпечити безперервне оновлення моделей, їх стабільний інференс, централізований моніторинг та інтуїтивно зрозумілий інтерфейс користувача. Її компоненти взаємодіють через стандартизовані механізми, що робить систему гнучкою, реконфігурованою та придатною до розгортання у будь-яких хмарних середовищах.

3.2 Підхід до реалізації контейнеризованих модулів

Реалізація системи автоматизації розгортання ШІ-застосунків спирається на принципи контейнеризації, які визначили спосіб побудови архітектури, логіку взаємодії між сервісами та механізми їх оновлення. Контейнеризація стала ключовим рішенням, оскільки дозволила досягти передбачуваності виконання, максимальної портативності, ізоляції компонентів та незалежного масштабування окремих модулів. У цьому підрозділі описано загальний підхід до побудови контейнеризованого середовища, вибір інструментів, структуру Docker-образів і принципи організації мережевої та файлової взаємодії між сервісами системи.

Основною платформою для контейнеризації обрано Docker, що забезпечив стандартизоване середовище для кожного з модулів. Вибір Docker був обумовлений необхідністю створити універсальні, повторно відтворювані образи сервісів, які поводитимуться однаково на локальній машині розробника, у середовищі CI/CD та під час розгортання на сервері. Кожний сервіс системи — `ai_api`, `ai_trainer`, `ai_monitor` та `ai_frontend` — отримав власний Dockerfile, у якому визначено базовий образ Python [15], залежності, змінні оточення, оптимізації розміру та параметри виконання(рисунок 7).

```

FROM python:3.10-slim
# Робоча директорія
WORKDIR /app
# Системні утиліти для curl та pillow codecs
RUN apt-get update && \
    apt-get install -y --no-install-recommends build-essential curl libgl1 && \
    rm -rf /var/lib/apt/lists/*
# Копіюємо залежності
COPY requirements.txt .
# Встановлюємо залежності
RUN pip install --no-cache-dir -r requirements.txt
# Копіюємо увесь код
COPY . .
# Створюємо папку логів
RUN mkdir -p /app/logs
# Відкриваємо порт
EXPOSE 8080
# Healthcheck
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD curl -f http://127.0.0.1:8080/health || exit 1
# Використовуємо gunicorn як production server; модуль: src.api:app
CMD ["gunicorn", "-w", "2", "-b", "0.0.0.0:8080", "src.api:app"]

```

Рисунок 7 - Dockerfile для сервісу ai_api

Важливою характеристикою побудови контейнерів у цій системі є мінімалістичність та оптимізація. Для всіх модулів використано базові образи python:3.10-slim [15], які забезпечують невеликий розмір, швидке завантаження та зменшення часу на побудову CI/CD-пайплайна. Внутрішнє середовище контейнерів зібрано з урахуванням можливостей подальшого масштабування, ізоляції залежностей та забезпечення стабільності виконання. Завдяки цьому система не залежить від локальних бібліотек і конфігурацій сервера, що має вирішальне значення для хмарних розгортань.

Особливу увагу під час формування контейнерів було приділено підтримці графічного процесора для сервісу тренування ai_trainer. Для цього Docker-контейнери інтегровано з NVIDIA Container Toolkit, що дозволяє передавати

доступ до GPU через механізм driver: nvidia та використовувати апаратне прискорення для навчання моделей (рисунок 8).

```
ai_trainer:
  build: ./trainer
  container_name: ai_trainer
  ports:
    - "8090:8090"
  restart: on-failure
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: all
            capabilities: [gpu]
  environment:
    - NVIDIA_VISIBLE_DEVICES=all
  volumes:
    - ./models:/models
    - ./trainer_logs:/trainer_logs
```

Рисунок 8 - Надання контейнеру ai_trainer доступ до GPU

Таким чином, середовище не прив'язане до конкретного типу апаратного забезпечення, а сервіс визначає GPU автоматично через PyTorch (рисунок 9).

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Рисунок 9 - Автоматичне визначення типу апаратного забезпечення

Ще одним важливим елементом архітектури контейнеризованих модулів стали спільні томи Docker, які забезпечують персистентність даних. Ключове сховище ./models/ змонтоване в контейнерах ai_api та ai_trainer для обміну моделями та метаданими тренування (Рис. 10).

```
volumes:
  - ./models:/models
```

Рисунок 10 - Спільне сховище для обміну моделями та метаданими тренування

Такий підхід дозволяє сервісу інференсу завжди мати доступ до актуальної моделі, водночас зберігаючи всі попередні версії для можливого відкату або аналізу. Використання зовнішніх томів також забезпечує стійкість даних у випадку перезапуску контейнерів або оновлення сервісів.

Організація мережевої взаємодії між контейнерами реалізована через внутрішню мережу Docker Compose [4], що надає сервісам статичні DNS-імена та спрощує конфігурацію REST-запитів у коді (Рис 11).

PORTS	NAMES
0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp	ai_trainer
0.0.0.0:8070->8070/tcp, [::]:8070->8070/tcp	ai_monitor
0.0.0.0:8050->8050/tcp, [::]:8050->8050/tcp	ai_frontend
0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	ai_api_blue

Рисунок 11 - Організація мережевої взаємодії між контейнерами

Завдяки цьому різні модулі можуть звертатися один до одного через прості URI-адреси, наприклад `http://ai_trainer:8090/status`, незалежно від зовнішньої структури серверів. Подібний підхід гарантує стабільність взаємодії між компонентами навіть під час оновлень, пересоздань контейнерів або зміни середовища виконання.

Система також використовує Docker Compose для оркестрації контейнерів, що дозволяє гнучко керувати запуском серверів blue та green під час оновлень. Кожний стек визначено окремим файлом конфігурації (`docker-compose.blue.yml` та `docker-compose.green.yml`), що відокремлює оточення двох версій сервісу інференсу та дозволяє безболісно перемикатися між ними (рисунки 12 - 13).

```

ai_api_blue:
  build: ./app
  container_name: ai_api_blue
  environment:
    - DEPLOY_COLOR=blue
  ports:
    - "8080:8080"
  restart: always
  healthcheck:
    test: ["CMD", "curl", "-f", "http://127.0.0.1:8080/health"]
    interval: 30s
    timeout: 5s
    retries: 3
  volumes:
    - ./models:/models
    - ./logs:/logs

```

Рисунок 12 - Файл docker-compose для оркестрації сервера ai_api_blue

```

services:
  ai_api_green:
    build: ./app
    container_name: ai_api_green
    environment:
      - DEPLOY_COLOR=green
    ports:
      - "8080:8080"
    restart: always
    healthcheck:
      test: ["CMD", "curl", "-f", "http://127.0.0.1:8080/health"]
      interval: 30s
      timeout: 5s
      retries: 3
    volumes:
      - ./models:/models
      - ./logs:/logs

```

Рисунок 13 - Файл docker-compose для оркестрації сервера ai_api_green

У процесі оновлення CI/CD-система активує скрипт розгортання, який визначає поточний стек, піднімає альтернативний, виконує health-check нової версії та, у разі успіху, перемикає трафік на новий контейнер (Рис. 14).

```

if docker ps | grep -q "ai_api_blue"; then
    ACTIVE="blue"
    NEXT="green"
elif docker ps | grep -q "ai_api_green"; then
    ACTIVE="green"
    NEXT="blue"
else
    ACTIVE="none"
    NEXT="blue"
fi

echo "Active environment: $ACTIVE"
echo "Next environment:  $NEXT"

echo "=== Building next version ($NEXT) ==="
docker compose -f docker-compose.$NEXT.yml build

if [ "$ACTIVE" != "none" ]; then
    echo "Stopping OLD ($ACTIVE) stack..."
    docker compose -f docker-compose.$ACTIVE.yml down
fi

echo "=== Starting $NEXT stack ==="
docker compose -f docker-compose.$NEXT.yml up -d

```

Рисунок 14 - Скрипт зміни робочого серверу інференс сервісу

Таким чином, підхід до реалізації контейнеризованих модулів у розробленій системі забезпечує високу портативність, ізоляцію залежностей, можливість автоматичного оновлення та підтримку масштабування. Контейнеризація створила передумови для впровадження DevOps-та MLOps-практик, дозволила впорядкувати життєвий цикл моделі, а також стала основою для реалізації механізму безперервного розгортання, що є ключовим аспектом у хмарних інтелектуальних системах.

3.3 Реалізація автоматичного тренування нейронних моделей

Механізм автоматичного тренування нейронних моделей у розробленій системі реалізовано у вигляді окремого сервісу `ai_trainer`, який функціонує як самостійний контейнеризований модуль із власним програмним інтерфейсом. Його основне завдання полягає у проведенні навчання моделі, збереженні результатів та ініціюванні оновлення сервісу інференсу без необхідності зупинки

всієї системи. Такий підхід відповідає концепції MLOps, у межах якої тренування розглядається не як разова операція, а як регулярний, автоматизований процес, інтегрований у загальний життєвий цикл моделі.

Архітектурно сервіс `ai_trainer` побудований за принципом розділення відповідальностей: логіка обробки HTTP-запитів виділена в окремий модуль програмного інтерфейсу (API), тоді як безпосередні обчислення, пов'язані з навчанням нейронної мережі, реалізовано у спеціальному модулі тренування. Узагальнену структуру сервісу та взаємодію між основними компонентами зображено на рисунку 15.

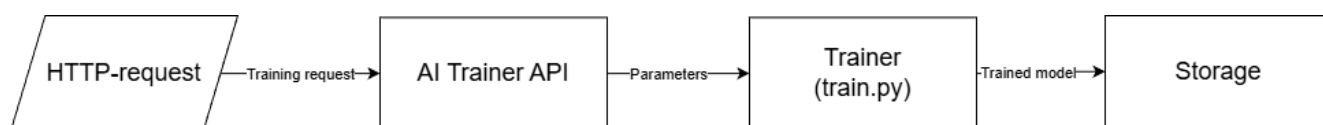


Рисунок 15 - Узагальнена структура сервісу `ai_trainer`

На схемі показано, як запит на запуск тренування надходить до API-сервісу, передається у функцію тренування, після чого результати записуються у спільні файлові сховища, а сервіс інференсу повідомляється про наявність нової версії моделі.

Ключовим елементом реалізації є вибір бібліотеки PyTorch для роботи з нейронними мережами. Це забезпечило гнучкість у побудові та модифікації архітектури моделі, а також зручний механізм роботи з графічним процесором. У фрагменті коду (рисунок 9) продемонстровано типовий спосіб визначення обчислювального пристрою. Під час запуску тренування сервіс перевіряє наявність GPU та у разі його доступності переносить модель і дані на графічний процесор. У разі відсутності апаратного прискорення тренування виконується на CPU. Така універсальність є важливою умовою для систем, що можуть розгортатися як на локальних серверах, так і в хмарних середовищах з різною конфігурацією обладнання.

Безпосередньо для задачі класифікації зображень у системі використано архітектуру ResNet-18, що є компактною, проте достатньо потужною для роботи з датасетом CIFAR-10.

На рисунку 16 умовно показано загальну структуру етапу тренування.

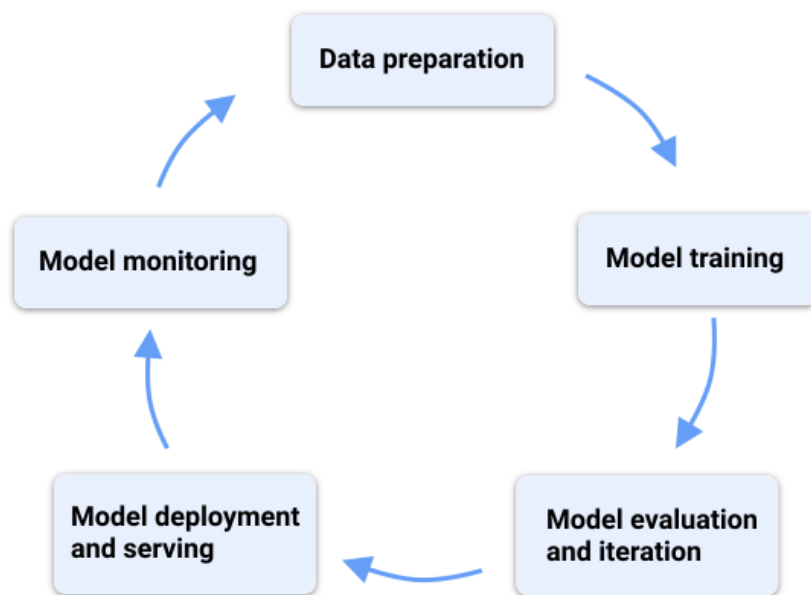


Рисунок 16 - Загальна структура етапу тренування моделі сервісу ai_trainer

Така схема відображає типовий цикл навчання нейронної мережі, адаптований до умов контейнеризованого середовища.

Окрему увагу в реалізації приділено збереженню результатів тренування. Після завершення навчання модель записується у спільний персистентний том `./models`, що змонтований як у контейнері `ai_trainer`, так і у контейнері сервісу інференсу `ai_api`.

На рисунку 17 наведено фрагмент коду функцій збереження, яка створює як версіонований файл моделі з часовою міткою, та оновлення спеціального файлу актуальної версії.

```
def save_model_state(state_dict):
    ts = datetime.utcnow().strftime("%Y%m%d_%H%M%S")
    name = f"model_v{ts}.pth"
    path = os.path.join(MODELS_DIR, name)
    return path, name

def set_latest(path):
    latest = os.path.join(MODELS_DIR, "model_latest.pth")
    try:
        if os.path.exists(latest):
            os.remove(latest)
        shutil.copy2(path, latest)
    except Exception:
        shutil.copy2(path, latest) # fallback
```

Рисунок 17 - Функції збереження та оновлення файлів моделей

Це дозволяє одночасно підтримувати історію модельних артефактів та мати єдину «поточну» модель, яку використовує сервіс інференсу.

Крім ваг моделі, у процесі тренування реєструються метрики якості та службова інформація.

Відповідний фрагмент коду показано на рисунку 18.

```
metadata = {
    "model_name": name,
    "accuracy": round(accuracy, 4),
    "loss": round(val_loss, 4),
    "epochs": epochs,
    "device": str(DEVICE),
    "train_time_sec": total_time,
    "finished_at": time.strftime("%Y-%m-%d %H:%M:%S")
}
```

Рисунок 18 - Фіксація метрик тренування моделей

У спеціальний файл метаданих, що розміщується у каталозі `./trainer_logs`, записуються значення точності, функції втрат, кількості епох, дата та час тренування та інші параметри. Це спрощує подальший аналіз результатів, дозволяє відстежувати динаміку навчання та використовувати ці дані у модулі моніторингу та візуалізації.

Важливою особливістю реалізації є автоматична інтеграція тренування з сервісом інференсу. Після успішного завершення навчання `ai_trainer` формує HTTP-запит до спеціального ендпоінта `/reload` сервісу `ai_api`. Фрагмент коду, що забезпечує цей процес відображено на рисунку 19.

```
@app.route("/reload", methods=["POST"])
def reload_route():
    global model_loaded
    try:
        ok = load_model(force_reload=True)
        model_loaded = bool(ok)
        logging.info(f"/reload called - model_loaded={model_loaded}")
        return jsonify({"reloaded": model_loaded})
    except Exception as e:
        logging.exception("Error loading model")
        return jsonify({"error": str(e)}), 500
```

Рисунок 19 - Запит на завантаження оновленого файлу натренованої моделі

Сервіс інференсу, отримавши запит, завантажує оновлений файл моделі з каталогу `./models` та замінює попередню версію. Завдяки такій взаємодії оновлення моделі виконується без перезапуску контейнера і без зміни мережових налаштувань, що відповідає вимогам безперервної роботи у хмарних середовищах.

Для контролю стану тренувального процесу сервіс `ai_trainer` надає окремий ендпоінт, через який можна отримати інформацію про останнє тренування, зокрема метрики якості та статус виконання. Цей інтерфейс використовується модулем моніторингу `ai_monitor`.

На рисунку 20 подано фрагмент коду відповідного обробника запитів, який зчитує дані з файлу стану та повертає їх у структурованому форматі.

```

state = {
    "status": "idle",      # idle / training / done / failed / cancelled
    "progress": 0.0,       # 0..100
    "epoch": 0,
    "accuracy": None,
    "loss": None,
    "started_at": None,
    "finished_at": None,
    "last_error": None
}

def set_state(**kwargs):
    with _lock:
        state.update(kwargs)

def get_state():
    with _lock:
        return dict(state)

def reset_state():
    with _lock:
        state.update({
            "status": "idle",
            "progress": 0.0,
            "epoch": 0,
            "accuracy": None,
            "loss": None,
            "started_at": None,
            "finished_at": None,
            "last_error": None
        })

```

Рисунок 20 - Обробка та повернення структурованих даних про тренування

Таким чином, користувач або адміністратор системи може оцінити ефективність останнього навчання та прийняти рішення про повторний запуск тренування або зміну параметрів.

У сукупності описані механізми формують завершений конвеєр автоматичного тренування нейронної моделі, який повністю інтегрований у контейнеризовану архітектуру системи. Сервіс `ai_trainer` не лише виконує обчислювальну задачу навчання, але й відповідає за коректне збереження результатів, ведення метаданих, забезпечення сумісності з сервісом інференсу та надання доступу до інформації про процес тренування іншим компонентам. Такий підхід дозволяє розглядати тренування не як допоміжну операцію, а як рівноправний, автономний елемент виробничої інфраструктури, що відповідає сучасним вимогам до MLOps-систем [18].

3.4 Налаштування CI/CD-конвеєра для автоматичного оновлення моделей

Впровадження механізму автоматичного оновлення моделей та сервісів у розробленій хмарно-орієнтованій системі ґрунтується на інтеграції інфраструктури контейнеризації з повноцінним CI/CD-конвеєром. Цей конвеєр забезпечує безперервне розгортання, перевірку, оновлення і контроль працездатності компонентів системи, що дозволяє мінімізувати ризик людських помилок та забезпечити стабільність сервісу інференсу навіть під час активних змін моделі або застосунку. Наявність автоматизованого конвеєра є важливою складовою MLOps-підходу [18], оскільки дозволяє зв'язати процес тренування моделі з її розгортанням у виробничому середовищі.

Основу CI/CD-процесу становить GitHub Actions — хмарний сервіс автоматизації, який забезпечує виконання визначеної послідовності дій при внесенні змін до репозиторію [7]. Для проєкту створено спеціальний workflow-файл, структура якого наведена на рисунку 21.

```
name: CI/CD Deploy

on:
  push:
    branches: ["main"]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build all services
        run: |
          docker compose -f docker-compose.yml build
```

Рисунок 21 - Файл deploy.yml, що забезпечує виконання визначеної послідовності дій при внесенні змін до репозиторію

У цьому файлі визначено події-тригери, середовище виконання, послідовність кроків для побудови Docker-образів та взаємодії з віддаленим сервером, на якому розгорнуто систему.

Після активації конвеєра GitHub Actions виконує побудову оновлених контейнерних образів усіх сервісів, зокрема `ai_api`, `ai_trainer`, `ai_monitor` та `ai_frontend`. Завдяки використанню мінімалістичних `Dockerfile` та кешування залежностей побудова образів відбувається ефективно, що позитивно впливає на швидкість оновлень.

Побудовані образи передаються на сервер, де відбувається основна частина автоматизації. На відміну від інфраструктур із застосуванням Kubernetes чи Docker Swarm, ця система використовує локальний підхід до оркестрації, в якому логіка перемикавання між `blue` і `green` конфігураціями реалізована у спеціальному скрипті `deploy.sh`. Фрагмент цього скрипта, наведений на рисунку 14, демонструє механізм визначення активної версії сервісу інференсу, що працює на сервері в даний момент.

Ключовою особливістю реалізації є те, що сині та зелені стекі не працюють одночасно, як це часто реалізується у багатосерверних або хмарних середовищах. У цьому проєкті використовується модель послідовного перемикавання: активний контейнер зупиняється, після чого запускається альтернативний, але на тому самому порту, що забезпечує прозорість для користувача. Внутрішні порти контейнерів залишаються стабільними, а зовнішній порт 8080 постійно відображає лише активну версію сервісу `ai_api`. Ця особливість є важливою для сценарію локального тестування та розгортання на одному сервері.

На наступному етапі скрипт виконує підняття резервної версії (наприклад, `green`), перевірку її працездатності через ендпоїнт `/health` та перемикавання трафіку. У разі успішного `health-check` старий стек зупиняється, а новий стає активним. Такий механізм забезпечує максимальну доступність сервісу інференсу, навіть попри те, що перемикавання відбувається шляхом перезапуску контейнера у рамках тестування.

З технічної точки зору важливо, що CI/CD не обмежується оновленням контейнерів. Він також гарантує:

- фіксовану процедуру розгортання;
- автоматичне виконання health-check;
- контроль версій та історії оновлень;
- прозоре документування всіх операцій через логування GitHub Actions;
- відсутність необхідності безпосереднього втручання оператора.

У разі подальшого масштабування системи CI/CD-конвеєр, створений у цьому проєкті, може бути легко адаптований до більш складних DevOps-середовищ. Наприклад, при перенесенні системи у Kubernetes CI/CD може виконувати застосування манифестів, керування Deployment-ресурсами, створення стратегій оновлення та автоматичне моделювання blue–green або canary-підходів на рівні кластерних сервісів.

Таким чином, реалізований CI/CD-конвеєр виступає ключовим елементом системи, що забезпечує автоматизацію розгортання, контроль цілісності інфраструктури та можливість безперервного оновлення моделей. Його інтеграція з контейнеризованими сервісами дає змогу розглядати всю систему як єдиний MLOps-конвеєр, у якому реалізовано повний цикл роботи моделі: від тренування до інференсу, розгортання та моніторингу.

3.5 Реалізовані способи моніторингу розгортання та тренування

Ефективний моніторинг є невід’ємною складовою будь-якої сучасної системи автоматизованого розгортання, особливо якщо йдеться про інтелектуальні сервіси, що працюють у контейнеризованому середовищі та підтримують безперервне оновлення моделей. Реалізована хмарно-орієнтована система включає окремий модуль моніторингу `ai_monitor`, який забезпечує спостережуваність за роботою всіх основних компонентів: сервісу інференсу `ai_api`, сервісу тренування `ai_trainer`, контейнерної інфраструктури та апаратних ресурсів. У цьому розділі розглядаються механізми, які дозволяють оцінювати

працездатність системи, виявляти потенційні проблеми та аналізувати ефективність тренувальних процесів.

Сервіс моніторингу реалізовано як легковаговий Flask-додаток [6], що запускається у власному контейнері й функціонує незалежно від інших сервісів. Він складається з підсистеми збору метрик, підсистеми перевірки стану сервісів, підсистеми логування та HTTP-інтерфейсу для отримання агрегованої інформації.

Завдяки ізольованості ai_monitor не впливає на продуктивність основних сервісів системи. Навіть у разі підвищеного навантаження на інференс або тренування моніторинговий модуль продовжує функціонувати стабільно та виконувати свою роль у загальній інфраструктурі.

Однією з ключових функцій моніторингового сервісу є періодичний збір інформації про використання апаратних ресурсів сервера. Для цього використовується бібліотека psutil [14], яка надає доступ до даних про:

- рівень завантаження процесора,
- обсяг використаної та доступної оперативної пам'яті,
- стан файлової системи,
- доступний простір на дискових томах,
- кількість активних процесів.

Процес збору метрик відбувається з певним інтервалом, який заданий у конфігурації системи. На рисунку 22 та рисунку 23 подано програмний код, що відображає періодичність збору та обробки системних параметрів.

```
def background_loop():  
    while True:  
        m = collect_metrics()  
        write_metrics(m)  
        time.sleep(15)
```

Рисунок 22 - Періодичний збір метрик

```
def collect_metrics():  
    metrics = {  
        "timestamp": time.strftime("%Y-%m-%d %H:%M:%S"),  
        "cpu_usage": psutil.cpu_percent(),  
        "ram_usage": psutil.virtual_memory().percent,  
        "disk_usage": psutil.disk_usage('/').percent,
```


Рисунок 23 - Обробка системних параметрів

Зібрані дані формують загальне уявлення про стан інфраструктури та можуть використовуватися як для ручної діагностики, так і для інтеграції з зовнішніми системами візуалізації.

Другим важливим напрямком моніторингу є контроль стану ключових сервісів системи — `ai_api` та `ai_trainer`.

Сервіс моніторингу періодично надсилає HTTP-запити до спеціальних діагностичних ендпоїнтів:

- `/health` у `ai_api`,
- `/metrics` або `/state` у `ai_trainer`.

Результати цих запитів дають змогу оцінити:

- працездатність моделей інференсу,
- наявність помилок у завантаженні моделі,
- готовність сервісу до оброблення запитів,
- прогрес тренування,
- точність та функцію втрат останньої епохи.

На рисунку 24 наведено умовний фрагмент логіки обробки відповідей від цих сервісів.

```
for name, url in SERVICES.items():
    metrics["services"][name] = get_service_status(name, url)

return metrics
```

Рисунок 24 - Збір та обробка метрик сервісів системи

Якщо якийсь сервіс тимчасово недоступний, `ai_monitor` фіксує цей факт у логах та повідомляє про нього через веб-інтерфейс системи.

Моніторинговий модуль формує не тільки поточний знімок стану системи, але й історію змін. З цією метою він зберігає останні `N` записів у персистентному

файлі `metrics.json`, що розташований у спеціальному каталозі логів. Структура цього файлу та принцип зберігання даних продемонстровані на рисунку 25.

```
def write_metrics(new_metrics):  
    with file_lock:  
        data = []  
        if os.path.exists(METRICS_FILE):  
            try:  
                with open(METRICS_FILE, "r") as f:  
                    data = json.load(f)  
            except:  
                data = []  
  
        data.append(new_metrics)  
        data = data[-200:]  
  
        with open(METRICS_FILE, "w") as f:  
            json.dump(data, f, indent=4)
```

Рисунок 25 - Зберігання зібраних метрик у файлі `metrics.json`

Така історичність дозволяє:

- аналізувати динаміку навантаження,
- відстежувати зміни після оновлення моделі,
- порівнювати продуктивність `blue` та `green` версій сервісу інференсу,
- виявляти можливі деградації продуктивності.

Це особливо важливо в умовах, коли модель регулярно оновлюється, і необхідно оцінювати вплив нової версії на систему.

Модуль моніторингу також надає відкритий HTTP-інтерфейс із двома ключовими ендпоінтами:

- `/dashboard` — агрегована версія останніх метрик у форматі, зручному для відображення у веб-інтерфейсі;

- /metrics_json — структуровані дані у форматі JSON, придатні для інтеграції з Grafana, Prometheus або сторонніми системами аналізу [9].

На рисунку 26 наведено приклад відповіді, яку повертає ендпоїнт /dashboard.

```

"monitor": {
  "cpu_usage": 2.8,
  "disk_usage": 3.4,
  "ram_usage": 22.1,
  "services": {
    "ai_api": {
      "data": {
        "deploy_color": "blue",
        "model_loaded": true,
        "model_version": "model_v20251126_085138.pth",
        "status": "ok",
        "uptime_seconds": 604
      },
      "ok": true,
      "status_code": 200
    },
    "ai_trainer": {
      "data": {
        "accuracy": null,
        "epoch": 0,
        "loss": null,
        "progress": 0,
        "status": "idle"
      },
      "ok": true,
      "status_code": 200
    }
  },
  "timestamp": "2025-11-27 09:11:09"
}

```

Рисунок 26 - Приклад відповіді від сервісу ai_monitor

Ці дані використовуються веб-інтерфейсом ai_frontend для формування повідомлень користувачеві.

Таким чином, модуль ai_monitor виконує роль центрального вузла спостережуваності, через який оператор може отримати повну картину роботи системи в режимі реального часу.

Модуль моніторингу ai_monitor є важливою складовою розробленої системи. Він забезпечує:

- детальний контроль навантаження та ресурсів,
- відстеження працездатності сервісів інференсу та тренування,

- фіксацію та аналіз тренувальних метрик,
- формування історії системного стану,
- інтеграцію з веб-інтерфейсом.

Усе це робить систему прозорою, передбачуваною та безпечною для регулярних оновлень моделей і компонентів. Моніторинг стає не допоміжним інструментом, а повноцінним елементом інфраструктури, який підтримує життєздатність і керованість розробленої хмарно-орієнтованої системи.

3.6 Графічний інтерфейс користувача

Графічний інтерфейс користувача є необхідним елементом розробленої хмарно-орієнтованої системи, оскільки забезпечує зручний доступ до функціоналу всіх основних компонентів: сервісу інференсу, тренування нейромережі, модуля моніторингу та CI/CD-процесів. Веб-інтерфейс виконує роль інтеграційної панелі, через яку користувач взаємодіє з автономними сервісами системи, не потребуючи знань про внутрішню архітектуру, файлову структуру чи REST API.

Графічний інтерфейс реалізовано у вигляді окремого контейнеризованого модуля `ai_frontend`, що працює незалежно від інших компонентів і спілкується з ними через HTTP-запити. В основі лежить Flask, що забезпечує легковаговість, простоту конфігурування та мінімальний час відгуку [6].

Реалізований графічний інтерфейс користувача зображений на рисунку 27.

AI Deploy Dashboard

Проста панель для роботи з моделлю: інференс, тренування, моніторинг.

1. Класифікація зображення

Вибрати файл

Файл не вибрано

Надіслати на інференс

Результат:

–

2. Запуск тренування

Epochs: 1

Batch size: 64

LR: 0,001

Запустити тренування

Статус тренування:

–

3. Статус системи

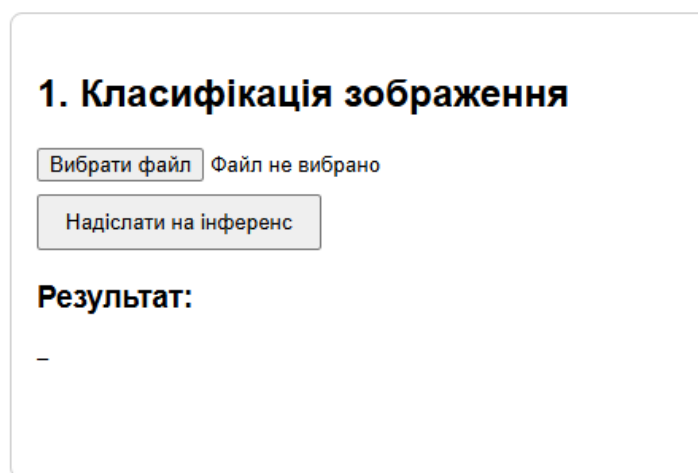
Оновити статус

–

Рисунок 27 - Реалізований графічний інтерфейс користувача

Фронтенд-модуль не містить складної бізнес-логіки: його завдання полягає у надсиланні запитів до відповідних сервісів та відображенні отриманих даних у зручному для користувача вигляді. Взаємодія з мікросервісами здійснюється через `host.docker.internal`, що дає змогу звертатися до контейнерів за стабільними адресами незалежно від внутрішньої мережі Docker. Це особливо зручно на локальних або гібридних розгортаннях.

Однією з основних функцій веб-інтерфейсу є можливість надсилання зображень до сервісу інференсу та отримання результатів класифікації. На рисунку 28 подано загальний вигляд сторінки класифікації.



1. Класифікація зображення

Вибрати файл Файл не вибрано

Надіслати на інференс

Результат:

–

Рисунок 28 - Розділ класифікації зображень

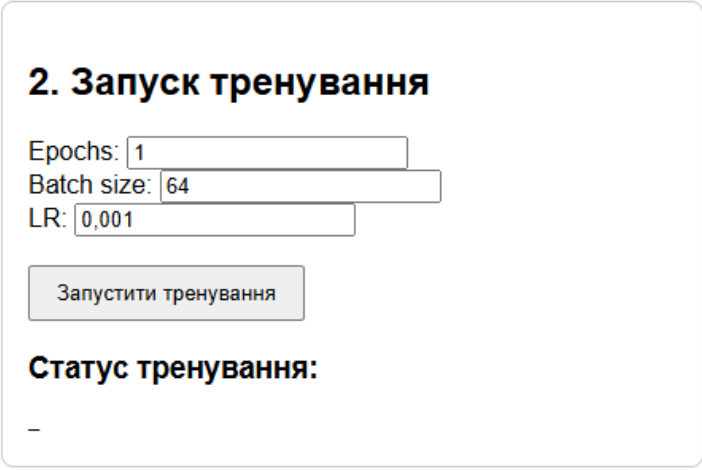
Користувач завантажує файл зображення за допомогою стандартного елемента введення, після чого веб-інтерфейс формує POST-запит до ендпоїнта `/predict` сервісу `ai_api`. Результат класифікації — передбачений клас та ймовірність — відображається на екрані разом з деталями обробки.

Такий механізм дозволяє інтуїтивно протестувати актуальну версію моделі без необхідності роботи з REST API або зовнішніми інструментами.

Для керування процесом тренування у веб-інтерфейсі реалізовано окремий розділ, де користувач може:

- вибрати кількість епох;
- ініціювати запуск тренування;
- переглядати метрики останнього навчання;
- контролювати стан сервісу тренування.

Загальний вигляд цієї частини інтерфейсу зображено на рисунку 29.



2. Запуск тренування

Epochs:

Batch size:

LR:

Статус тренування:

-

Рисунок 29 - Розділ керування тренуваннями нейромережі

Після натискання кнопки «Запустити тренування» фронтенд формує запит до ендпоїнта `/train` сервісу `ai_trainer`.

Після завершення навчання система відображає у веб-інтерфейсі отримані метрики точності, втрат та інші параметри, які зберігаються у каталозі `trainer_logs`. Це дозволяє користувачу легко контролювати якість моделі та приймати рішення щодо повторного навчання чи зміни параметрів.

Однією з ключових функцій інтерфейсу є можливість у режимі реального часу переглядати стан системи. На рисунку 30 наведено приклад панелі моніторингу, що містить:

- використання CPU,
- обсяг спожитої пам'яті,
- стан дискових ресурсів,
- результат health-check сервісу інференсу,
- прогрес тренування моделі,
- історію метрик з файлу `metrics.json`.

3. Статус системи

Оновити статус

```
{
  "ai_api": {
    "deploy_color": "blue",
    "model_loaded": true,
    "model_version": "model_v20251126_085138.pth",
    "status": "ok",
    "uptime_seconds": 604
  },
  "ai_trainer": {
    "accuracy": null,
    "epoch": 0,
    "finished_at": null,
    "last_error": null,
    "loss": null,
    "progress": 0,
    "started_at": null,
    "status": "idle"
  },
  "monitor": {
    "cpu_usage": 2.8,
    "disk_usage": 3.4,
    "ram_usage": 22.1,
    "services": {
      "ai_api": {
        "data": {
          "deploy_color": "blue",
          "model_loaded": true,
          "model_version": "model_v20251126_085138.pth",
          "status": "ok",
          "uptime_seconds": 604
        },
        "ok": true,
        "status_code": 200
      },
      "ai_trainer": {
        "data": {
          "accuracy": null,
          "epoch": 0,
          "loss": null,
          "progress": 0,
          "status": "idle"
        },
        "ok": true,
        "status_code": 200
      }
    },
    "timestamp": "2025-11-27 09:11:09"
  }
}
```

Рисунок 30 - Приклад панелі моніторингу системи

Дані отримуються через ендпоїнт `/dashboard` сервісу `ai_monitor`. Інтерфейс регулярно оновлює інформацію динамічно, що дозволяє відстежувати стан інфраструктури без перезавантаження сторінки.

Хоча інтерфейс не виконує безпосередньо розгортання системи, він відображає стан активної версії сервісу інференсу. У випадку blue–green оновлення користувач може побачити:

- яка версія сервісу активна,
- чи коректно завантажилася модель та її версія,
- час активності.

На рисунку 31 продемонстровано приклад відображення версії активного стеку у веб-інтерфейсі.

```

    "ai_api": {
      "deploy_color": "blue",
      "model_loaded": true,
      "model_version": "model_v20251126_085138.pth",
      "status": "ok",
      "uptime_seconds": 604
    }
  
```

Рисунок 31 - Приклад відображення активної версії ai_api

Це підвищує прозорість системи та дозволяє оператору контролювати процеси оновлення без перевірки журналів вручну.

Графічний інтерфейс користувача виконує низку важливих функцій:

- Забезпечує доступність системи для широкого кола користувачів.
- Замість роботи з командним рядком чи REST API оператор працює із зрозумілою веб-панеллю.
- Уніфікує взаємодію з модулем інференсу, тренування та моніторингу.
- Уся інформація та дії зібрані в одному місці.
- Сприяє прискоренню робочих процесів.
- Час між тренуванням моделі та її практичним тестуванням зведено до мінімуму.
- Забезпечує помітний рівень спостережуваності.
- Інтерфейс дозволяє контролювати стан інфраструктури та виявляти потенційні проблеми на ранніх етапах.
- У сукупності веб-інтерфейс не є лише допоміжним елементом, а виступає повноцінним компонентом системи, що забезпечує доступність та зручність управління розробленою хмарно-орієнтованою платформою автоматичного розгортання ШІ-моделей.

Висновки до розділу

У цьому розділі було детально розглянуто повну реалізацію хмарно-орієнтованої системи автоматизованого розгортання, оновлення, моніторингу та експлуатації моделей штучного інтелекту, що є центральним програмним результатом даного дослідження. Представлена система сформована за сервісно-орієнтованою архітектурою та використовує контейнеризацію для забезпечення ізольованості, відтворюваності та масштабованості кожного компонента.

Насамперед було охарактеризовано загальну структуру системи та визначено роль основних модулів: сервісу інференсу, тренувального модуля, сервісу моніторингу, веб-інтерфейсу та засобів автоматичного розгортання. Особливу увагу приділено їх взаємодії, схемі даних та логічним зв'язкам, що забезпечують безперервну роботу всієї системи. Запропонована структура підтверджує, що компоненти функціонують як цілісна екосистема, в якій кожен сервіс виконує чітко визначену роль.

Далі описано підхід до контейнеризації модулів системи. Застосування Docker та Docker Compose дало змогу сформувати модульну інфраструктуру, де кожен компонент працює в окремому середовищі, але водночас має стандартизований спосіб взаємодії з іншими мікросервісами. У цьому ж підрозділі було розглянуто реалізацію механізму blue-green оновлення, який забезпечує можливість безпечного розгортання нових версій сервісу інференсу з мінімальним впливом на кінцевого користувача. Важливо, що система передбачає можливість подальшого масштабування, а також адаптацію до більш комплексних оркестраційних платформ.

У наступному пункті наведено реалізацію механізму автоматичного тренування моделей. Тренувальний сервіс повністю відокремлено від логіки інференсу, що дає змогу виконувати навчання без впливу на роботу інших компонентів. Особливе значення має автоматичне збереження результатів тренування, ведення метаданих та ініціювання оновлення моделі в сервісі інференсу без необхідності його перезапуску. Це забезпечує високий ступінь автономності системи та відповідає сучасним вимогам MLOps-процесів.

Розглянуто реалізацію CI/CD-конвеєра, який забезпечує автоматичне розгортання компонентів системи та виконання оновлень після кожної зміни у репозиторії. Використання GitHub Actions у поєднанні зі скриптом `deploy.sh` дозволило створити відтворюваний, надійний і повністю автоматизований процес оновлення, що охоплює побудову образів, перевірку працездатності, перемикання між версіями та контроль стабільності сервісів. Такий підхід інтегрує DevOps-практики у загальну архітектуру системи, забезпечуючи відповідність вимогам безперервного розгортання.

А також було приділено увагу реалізованим засобам моніторингу. Окремий модуль `ai_monitor` здійснює постійний контроль за станом інфраструктури, використанням ресурсів, працездатністю сервісів інференсу та тренування, а також зберігає історію метрик. Такий рівень спостережуваності дає змогу оперативно виявляти відхилення в роботі системи, відстежувати ефективність роботи моделі та аналізувати вплив оновлень на продуктивність.

На завершення, було детально описано графічний інтерфейс користувача, який забезпечує доступ до ключових функцій системи в інтерактивному та зрозумілому вигляді. Інтерфейс слугує точкою входу до всіх модулів системи, дозволяючи виконувати класифікацію, запускати тренування, переглядати метрики моніторингу та відстежувати стан інфраструктури. Такий підхід робить систему зручною для використання як технічними спеціалістами, так і користувачами, які не мають глибокого досвіду роботи з інструментами DevOps або MLOps.

У сукупності реалізований комплекс програмних рішень демонструє завершену та практично застосовну хмарно-орієнтовану систему, яка об'єднує процеси інференсу, тренування, моніторингу та автоматичного розгортання моделей штучного інтелекту. Розроблена архітектура повністю узгоджується з сучасними вимогами до побудови AI-платформ і підтверджує ефективність застосування контейнеризації, DevOps- та MLOps-підходів у практичних системах штучного інтелекту.

4 ВИПРОБУВАННЯ РОЗРОБЛЕНОЇ СИСТЕМИ ТА АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

4.1 Тестування роботи розробленої системи

Тестування є однією з ключових фаз життєвого циклу будь-якої інформаційної системи, особливо якщо вона включає механізми автоматичного розгортання, оновлення моделей та адаптацію компонентів у режимі реального часу. Метою тестування розробленої хмарно-орієнтованої системи є підтвердження її функціональної коректності, стабільності роботи, відповідності технічним вимогам і здатності виконувати інференс та оновлення моделей.

Основною метою випробувань було:

- перевірити коректність контейнеризації та взаємодії між сервісами;
- визначити стабільність роботи сервісу інференсу після багаторазових оновлень;
- оцінити ефективність механізму автоматичного тренування;
- перевірити роботу CI/CD-конвеєра та механізму blue–green оновлення;
- оцінити роботу системи моніторингу під час навантаження;
- перевірити коректність роботи веб-інтерфейсу.

На початковому етапі було проведено тестування кожного контейнера: `ai_api`, `ai_trainer`, `ai_monitor` та `ai_frontend`.

Перевірялися такі характеристики:

- коректність побудови Docker-образів;
- працездатність контейнера після запуску;
- правильність підключення томів;
- робота мережевих інтерфейсів та доступність ендпоїнтів.

Для цього використовувалися стандартні команди `docker logs`, `docker ps`, а також запити до REST API. Результати показали, що кожен сервіс успішно ініціалізується, коректно підключає залежності, а Flask-додатки стартують без помилок. Результати коректно цього тестування можна спостерігати на раніше наведених рисунку 11, рисунку 26 та рисунку 27.

Сервіс ai_ari було протестовано за допомогою реальних зображень CIFAR-10 та довільних тестових даних.

Основними критеріями оцінки були:

- коректність передбачення класу,
- стабільність роботи при багаторазових запитах,

Протестовано кілька зображень, що були взяті поза межами тренувального та валідаційного датасету. Оскільки використовувана модель пройшла тренування лише в одну епоху, то вона допустила помилки на деяких екземплярах (рисунки 32-33).



Рисунок 32 - Вхідне зображення doggy.jpg для моделі тренуваної в 1 епоху

1. Класифікація зображення

Вибрати файл

doggy.jpg

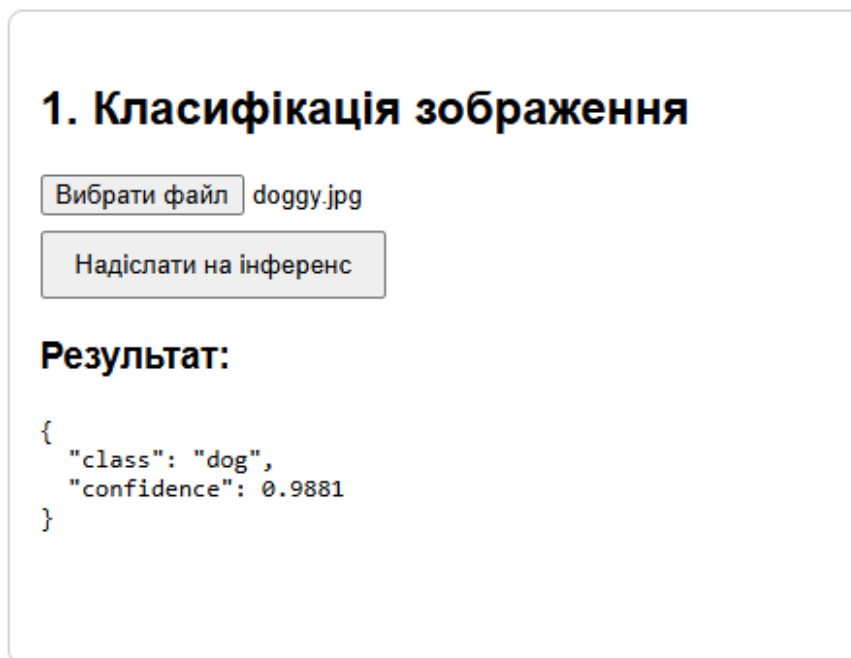
Надіслати на інференс

Результат:

```
{
  "class": "bird",
  "confidence": 0.5245
}
```

Рисунок 33 - Результат класифікації зображення doggy.jpg

Після тренування та використання нової моделі тренованої в 5 епох, були отримані задовільні результати(рисунки 34).



1. Класифікація зображення

Вибрати файл doggy.jpg

Надіслати на інференс

Результат:

```
{
  "class": "dog",
  "confidence": 0.9881
}
```

Рисунок 35 - Результат класифікації зображення doggy.jpg моделлю тренованою в 5 епох

Запити не викликали деградації продуктивності, що свідчить про стабільність обробки та правильну оптимізацію залежностей.

Сервіс ai_trainer було протестовано шляхом запуску тренування на одній та п'яти епохах. Усі експерименти мали такі цілі:

- перевірити коректність завантаження датасету;
- перевірити роботу GPU-прискорення, якщо доступно;
- оцінити швидкість тренування;
- перевірити збереження ваг моделі;
- перевірити коректність формування metadata-файлу;

У всіх тестових сценаріях тренування проходило без помилок(рисунки 36-38).

2. Запуск тренування

Epochs:
 Batch size:
 LR:

Статус тренування:

```
{
  "epochs": 1,
  "status": "started"
}
```

Рисунок 36 - Початок тренування моделі в одну епоху

```
"ai_trainer": {
  "accuracy": null,
  "epoch": 1,
  "finished_at": null,
  "last_error": null,
  "loss": 0.6842,
  "progress": 50,
  "started_at": 1764234694.6985562,
  "status": "training"
},
```

Рисунок 37 - Процес тренування моделі

```
"ai_trainer": {
  "accuracy": 0.8472,
  "epoch": 1,
  "finished_at": 1764235238.813838,
  "last_error": null,
  "loss": 0.4335,
  "progress": 100,
  "started_at": 1764234694.6985562,
  "status": "done"
},
```

Рисунок 38 - Закінчення тренування моделі

На рисунку 39 зображено завантаженість відеокарти, це означає що під час тренування нейромережевої моделі.

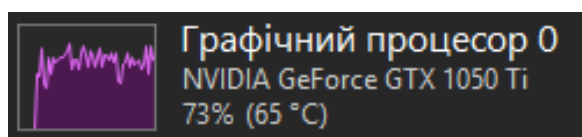


Рисунок 39 - Завантаженість GPU під час тренування моделі

Після завершення модель зберігалася у томі `./models` (рисунок 40).

```
model_latest.pth      model_v20251104_225135.pth  model_v20251127_092038.pth
model_v20251101_125225.pth  model_v20251121_021108.pth  training_metadata.json
model_v20251101_151753.pth  model_v20251126_085138.pth
```

Рисунок 40 - Збереженні моделі у томі `./models`

Та показники моделі коректно зберігалися в логах та відображалися у веб-інтерфейсі (рисунок 38).

Особливо важливою частиною випробувань було тестування CI/CD-процесу. Для цього виконувались послідовні коміти у гілку `main`, після яких автоматично:

- запускалися GitHub Actions;
- збиралися Docker-образи;
- виконувалася передача файлів та виклик `deploy.sh`;
- перемикався активний стек `blue ↔ green`;
- виконувався `health-check`.

На рисунку 41 представлено автоматичний результат від GitHub Actions після завантаження змін у віддалений репозиторій та збору Docker-образів.

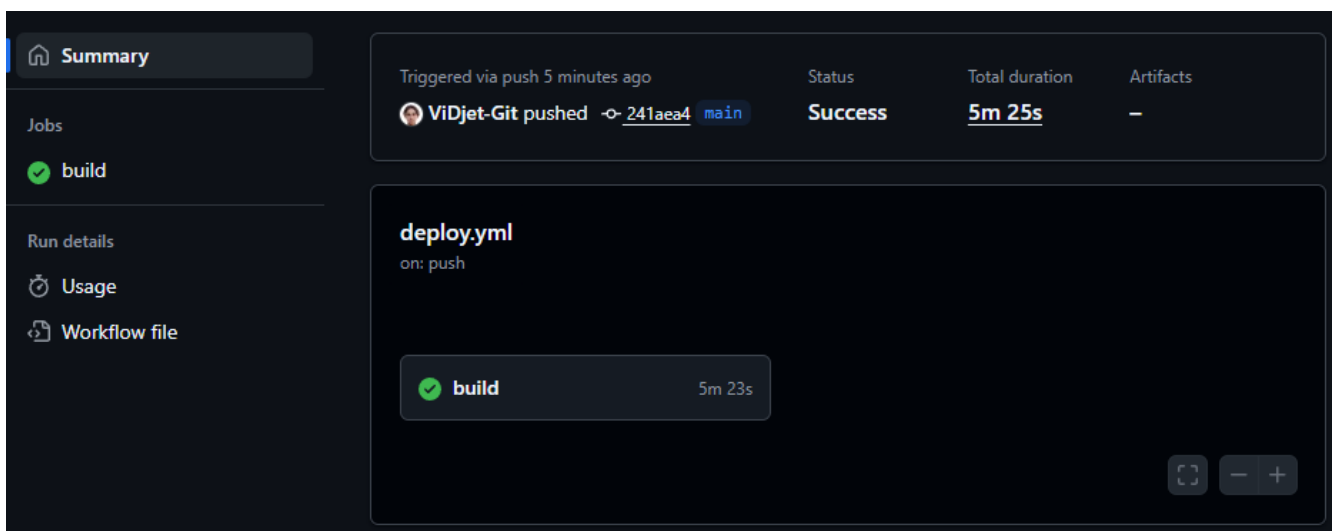


Рисунок 41 - Результати тестування та збирання Docker-образів від GitHub Actions

На рисунках 42-43 зображений результат після тренування моделі та виклику файлу `deploy.sh` для автоматичного зміни активного серверу `ai_api` та завантаження нетренованої моделі.

AI Deploy Dashboard

Проста панель для роботи з моделлю: інференс, тренування, моніторинг.

1. Класифікація зображення

Вибрати файл

Результат:

```
{
  "class": "cat",
  "confidence": 0.9892
}
```

2. Запуск тренування

Epochs:

Batch size:

LR:

Статус тренування:

```
{
  "epochs": 1,
  "status": "started"
}
```

3. Статус системи

```
{
  "ai_api": {
    "deploy_color": "blue",
    "model_loaded": true,
    "model_version": "model_v20251126_085138.pth",
    "status": "ok",
    "uptime_seconds": 944
  },
  "ai_trainer": {
    "accuracy": null,
    "epoch": 1,
    "finished_at": null,
    "last_error": null,
    "loss": 0.6842,
    "progress": 50,
    "started_at": 1764234694.6985562,
    "status": "training"
  },
}
```

Рисунок 42 - Стан серверів до закінчення тренування моделі

AI Deploy Dashboard

Проста панель для роботи з моделлю: інференс, тренування, моніторинг.

1. Класифікація зображення

cat.jpg

Результат:

```
{
  "class": "cat",
  "confidence": 0.9854
}
```

2. Запуск тренування

Epochs:
 Batch size:
 LR:

Статус тренування:

```
{
  "epochs": 1,
  "status": "started"
}
```

3. Статус системи

```
{
  "ai_api": {
    "deploy_color": "green",
    "model_loaded": true,
    "model_version": "model_v20251127_092038.pth",
    "status": "ok",
    "uptime_seconds": 27
  },
  "ai_trainer": {
    "accuracy": 0.8472,
    "epoch": 1,
    "finished_at": 1764235238.813838,
    "last_error": null,
    "loss": 0.4335,
    "progress": 100,
    "started_at": 1764234694.6985562,
    "status": "done"
  }
}
```

Рисунок 43 - Стан серверів після активації скрипта та оновлення моделі

Дані результати підтверджують коректність роботи сервісів та скриптів. Перемикання на інший сервер відбулося успішно, та модель, що використовується, теж оновилась. Це підтверджує правильність реалізації механізму оновлення та його відповідність поставленим вимогам.

Модуль `ai_monitor` теж було протестовано під час тестування інших сервісів. У результаті тестування було встановлено, що:

- частота запису значень у `metrics.json` залишається стабільною;
- система не пропускає запити і не створює зайвого навантаження;
- історія метрик формується коректно;
- веб-інтерфейс отримує актуальні дані у межах інтервалу оновлення.

У результаті жодних критичних помилок у роботі моніторингового модуля не було зафіксовано. Приклад відображення метрик можна побачити на рисунку 44.

3. Статус системи

Оновити статус

```
{
  "ai_api": {
    "deploy_color": "blue",
    "model_loaded": true,
    "model_version": "model_v20251126_085138.pth",
    "status": "ok",
    "uptime_seconds": 604
  },
  "ai_trainer": {
    "accuracy": null,
    "epoch": 0,
    "finished_at": null,
    "last_error": null,
    "loss": null,
    "progress": 0,
    "started_at": null,
    "status": "idle"
  },
  "monitor": {
    "cpu_usage": 2.8,
    "disk_usage": 3.4,
    "ram_usage": 22.1,
    "services": {
      "ai_api": {
        "data": {
          "deploy_color": "blue",
          "model_loaded": true,
          "model_version": "model_v20251126_085138.pth",
          "status": "ok",
          "uptime_seconds": 604
        },
        "ok": true,
        "status_code": 200
      },
      "ai_trainer": {
        "data": {
          "accuracy": null,
          "epoch": 0,
          "loss": null,
          "progress": 0,
          "status": "idle"
        },
        "ok": true,
        "status_code": 200
      }
    },
    "timestamp": "2025-11-27 09:11:09"
  }
}
```

Рисунок 44 - Відображення повного статусу системи у веб-інтерфейсі

Оскільки веб-інтерфейс `ai_frontend` у даному проєкті використовується як допоміжний інструмент для розуміння та тестування системи, у результаті він показав себе добре, забезпечивши коректне виконання заданих функцій.

У результаті проведених випробувань встановлено, що:

- усі компоненти системи функціонують коректно як окремо, так і в інтегрованому середовищі;
- механізм контейнеризації спрощує розгортання та забезпечує повторюваність середовища;
- система здатна виконувати інференс без простою, навіть під час оновлення моделі чи сервісів;
- автоматичне тренування і збереження моделей працює стабільно і надійно;
- CI/CD-конвеєр гарантує безперервне оновлення та швидкий перехід між версіями та серверами;
- модуль моніторингу дозволяє відстежувати стан системи в режимі реального часу;
- веб-інтерфейс забезпечує інтуїтивну взаємодію з усіма сервісами.

Таким чином, система повністю відповідає вимогам, поставленим на етапі проєктування, і демонструє високий рівень надійності, автономності та практичної застосовності.

4.2 Оцінка продуктивності, надійності та масштабованості системи

Оцінювання розробленої системи включало аналіз її ключових властивостей — продуктивності, надійності та масштабованості — з урахуванням особливостей контейнеризованої архітектури, механізмів автоматичного розгортання та інтегрованих процесів тренування моделей. Усі характеристики оцінювалися на основі фактичної роботи системи, поведінки її сервісів та результатів інтеграційного тестування.

Продуктивність оцінювалася переважно за здатністю сервісів стабільно виконувати свої функції та взаємодіяти між собою без збоїв. Проведені випробування показали, що:

- сервіс інференсу коректно обробляє запити класифікації навіть під час роботи інших компонентів;
- тренувальний модуль виконує навчання моделі без впливу на роботу інференсу;
- завантаження нової моделі через механізм `/reload` відбувається миттєво та без переривання роботи системи;
- веб-інтерфейс оперативно відображає актуальні дані від усіх сервісів.

Таким чином, система демонструє сталу продуктивність при одночасній роботі кількох модулів та правильне застосування контейнерної ізоляції.

У межах тестування було перевірено поведінку системи при типовому та підвищеному навантаженні, а також її здатність коректно відновлюватися після можливих збоїв. Отримані результати засвідчили:

- відсутність критичних помилок у роботі контейнерів під час перезапусків;
- збереження доступності сервісу інференсу під час тренування та оновлення моделі;
- стабільну роботу моніторингу, який продовжує фіксувати метрики незалежно від стану інших компонентів;
- коректну обробку некоректних або неповних даних, що надходять до API.

Механізми автоперезапуску контейнерів та розмежування функціональності між сервісами забезпечують високу загальну надійність системи.

Архітектурні рішення, прийняті під час розробки, дозволяють системі масштабуватися у кількох напрямках.

- Сервіс інференсу може мати кілька екземплярів, оскільки його логіка не залежить від конкретного середовища, а модель завантажується з

єдиного персистентного тому. Це робить можливим горизонтальне масштабування у хмарних платформах.

- Модуль тренування може бути перенесений на більш потужні сервери або середовища з GPU, що дає змогу збільшити швидкість навчання без зміни логіки застосування.
- Модуль моніторингу може бути інтегрований з повноцінними інструментами візуалізації, такими як Prometheus та Grafana, що відкриває можливість масштабування моніторингових функцій.
- CI/CD-конвеєр легко адаптується до кластерних інфраструктур, оскільки ґрунтується на стандартних інструментах автоматизації та не прив'язаний до конкретного середовища виконання.

На практиці це означає, що система може працювати як на односерверному розгортанні, так і у повноцінному хмарному середовищі з автоматичним балансуванням навантаження.

Проведений аналіз засвідчує, що розроблена система:

- підтримує стабільну роботу всіх модулів при різних умовах;
- зберігає працездатність навіть під час оновлення моделі або окремих сервісів;
- має високу надійність завдяки контейнеризації та відокремленню логіки між мікросервісами;
- демонструє потенціал до масштабування при переході на більш комплексні інфраструктури.

Таким чином, система відповідає вимогам до сучасних хмарних AI-платформ та може бути практично застосована для розгортання й обслуговування моделей машинного навчання у виробничих середовищах.

4.3 Порівняння отриманих результатів зі звичними підходами розгортання

Для оцінки ефективності розробленої системи доцільно порівняти її можливості з традиційними підходами до розгортання програмного забезпечення, які часто використовуються у невеликих проєктах, локальних та хмарних середовищах або у процесі розробки моделей без спеціалізованих DevOps/MLOps інструментів.

Традиційні підходи передбачають ручне розгортання моделей, залежність від середовища конкретної машини, відсутність автоматизованого оновлення та доволі обмежені засоби контролю стану сервісів. У таких сценаріях зміна версії моделі або оновлення застосунку часто вимагає повної зупинки системи, повторної інсталяції залежностей, ручної конфігурації та додаткового тестування.

Розроблена ж система працює за принципово іншою парадигмою. Завдяки використанню контейнеризації всі компоненти функціонують у стабільних, відтворюваних середовищах, що виключає ризики, пов'язані з конфігурацією операційної системи або залежностей. Наявність механізму автоматичного оновлення моделі через `/reload` робить можливим оновлення без зупинки сервісу інференсу, що традиційні підходи забезпечити не можуть [11].

Ще однією важливою відмінністю є інтеграція CI/CD. У класичних сценаріях переходи між версіями системи часто виконуються вручну, що збільшує ризики помилок. У розробленій системі автоматизований конвеєр повністю керує процесом побудови, перевірки та розгортання сервісів. Наявність blue-green механізму підвищує надійність оновлень і дозволяє перевірити нову версію перед тим, як вона стане основною.

Моніторингові можливості також вигідно відрізняють систему від традиційних рішень. Там, де зазвичай адміністратор змушений вручну відстежувати стан серверів або аналізувати журнали, у розробленій системі впроваджений окремий моніторинговий сервіс, що автоматично збирає метрики роботи інфраструктури, стан тренування та інференсу, а також відображає все у фронтенді в реальному часі [11].

З погляду масштабованості розроблена система має також значні переваги. Традиційні підходи зазвичай не передбачають легкого переходу до хмарних інфраструктур або кластеризованих середовищ [11]. Натомість контейнеризована архітектура дозволяє масштабувати окремі модулі незалежно, перенаправляти навантаження або переносити систему у середовища Kubernetes чи Docker Swarm без значних модифікацій.

Отже, у порівнянні з класичними підходами розгортання, розроблена система демонструє суттєві переваги, зокрема в автоматизації, надійності, масштабованості та відтворюваності. Це доводить, що застосування контейнеризації, DevOps- і MLOps-підходів суттєво підвищує ефективність роботи з моделями штучного інтелекту та робить процес розгортання більш передбачуваним і стабільним.

Висновки до розділу

У цьому розділі було проведено комплексне випробування розробленої хмарно-орієнтованої системи та здійснено аналіз отриманих результатів щодо її функціональності, ефективності та здатності до масштабування. Проведене тестування засвідчило, що всі компоненти системи працюють стабільно як окремо, так і в інтегрованому середовищі. Завдяки використанню контейнеризації забезпечено повторюваність середовища, ізолюваність модулів та відсутність взаємного впливу під час тренування, інференсу або оновлення.

Оцінювання продуктивності підтвердило, що система здатна виконувати ключові операції — класифікацію зображень, збереження та оновлення моделі, збір метрик, роботу фроненда — без збоїв та без значних затримок. Механізм автоматичного тренування моделі та її інтеграції у сервіс інференсу функціонує надійно, а використання окремого модуля моніторингу забезпечує постійний контроль за станом інфраструктури та поведінкою сервісів.

Запроваджений CI/CD-конвеєр успішно автоматизує процес розгортання, забезпечує безпечне оновлення сервісів та відповідає вимогам безперервної

доступності. Наявність механізму blue–green оновлення підтверджує здатність системи виконувати перехід між версіями без зупинки роботи інференсу.

Порівняння отриманих результатів із традиційними підходами розгортання показало суттєву перевагу запропонованої системи у частині автоматизації, відмовостійкості та масштабованості. Використання DevOps/MLOps-підходів, контейнерної архітектури та централізованого моніторингу робить систему стійкою, гнучкою та придатною для подальшого розвитку.

Таким чином, результати тестування підтверджують відповідність розробленої системи сучасним практикам побудови AI-платформ та її здатність ефективно виконувати функції автоматичного розгортання та обслуговування моделей штучного інтелекту у хмарному середовищі.

ВИСНОВКИ

У ході виконання магістерської роботи було розроблено, реалізовано та досліджено хмарно-орієнтовану систему автоматизації розгортання, оновлення та експлуатації моделей штучного інтелекту. Метою дослідження було створення програмної інфраструктури, яка поєднує сучасні підходи DevOps та MLOps із контейнеризацією та забезпечує повний цикл роботи моделі — від тренування до розгортання у виробничому середовищі.

Проведений аналіз теоретичних аспектів показав, що сучасні системи оброблення даних та машинного навчання потребують високого рівня автоматизації, надійної інфраструктури та чіткого розділення відповідальностей між компонентами. Ручні методи розгортання та підтримки моделей вже не відповідають вимогам до швидкості оновлень і стабільності сервісів. Це обумовило необхідність застосування контейнеризації, безперервних конвеєрів CI/CD та підходів MLOps, що забезпечують керованість, передбачуваність і масштабованість розроблених систем.

У практичній частині роботи було створено повноцінну мікросервісну архітектуру, що включає окремі контейнери для інференсу, тренування, моніторингу, веб-інтерфейсу та механізмів автоматизації. Сервіс інференсу відповідає за класифікацію зображень і підтримує миттєве оновлення моделі. Тренувальний модуль виконує навчання нейронної мережі, веде облік результатів, зберігає ваги моделей і автоматично ініціює оновлення сервісу інференсу. Модуль моніторингу здійснює постійний контроль стану системи та відображає реальні метрики роботи, що дозволяє швидко реагувати на можливі відхилення. Окремий веб-інтерфейс робить роботу з системою інтуїтивною, зрозумілою та доступною без додаткових інструментів.

Важливим результатом є створення CI/CD-конвеєра, що виконує автоматичне оновлення сервісу інференсу та інших компонентів системи. Реалізований механізм blue–green оновлення дає змогу впроваджувати нові версії моделей та сервісів без зупинки системи. Це особливо важливо для AI-рішень, де якість моделі може змінюватися часто, а доступність сервісу залишається

критичною. Фактично, побудована система формує повноцінний цикл безперервного вдосконалення AI-моделі — модель тренується, зберігається, оновлюється та миттєво інтегрується в інференс.

Проведені випробування підтвердили, що система працює стабільно та передбачувано навіть за умови активного оновлення моделей, одночасної роботи кількох модулів або підвищеного навантаження на тренування. Завдяки ізольованості контейнерів зміни в одному модулі не впливають на інші, а гнучкість інфраструктури дозволяє адаптувати систему до різних середовищ: від локального сервера до масштабованих хмарних платформ.

Порівняння з традиційними підходами до розгортання показало очевидні переваги розробленої системи. Вона мінімізує ручні операції, виключає залежність від конфігурації операційної системи, забезпечує миттєве оновлення моделей та дозволяє розгортати середовище будь-яку кількість разів із гарантованим однаковим результатом. Такі властивості роблять систему практично цінною та конкурентоспроможною в умовах реальних промислових сценаріїв.

У підсумку можна стверджувати, що поставлені у роботі цілі виконані повністю. Створено хмарно-орієнтовану систему автоматизації розгортання ШІ-застосунків, яка поєднує сучасні технології та методи розробки, демонструє високу стабільність, гнучкість та ефективність. Розроблена система може стати фундаментом для побудови більш складних AI-платформ, її архітектура легко розширюється, а функціонал може бути доповнений новими моделями, методами моніторингу та засобами оркестрації.

Важливо й те, що система не лише вирішує конкретну прикладну задачу — автоматизацію розгортання моделей машинного навчання — але й формує універсальний підхід до побудови інфраструктури AI-рішень. Результати роботи можуть бути корисними для широкого спектра завдань: від навчальних досліджень до промислових середовищ, де оновлення моделей і безперервність сервісів мають вирішальне значення.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Blue-Green Deployment Best Practices. Clever Cloud. URL: <https://www.clever.cloud/developers/doc/best-practices/blue-green/> (дата звернення 15.08.2025).
2. Deza A., Gift N. Practical MLOps: Operationalizing Machine Learning Models. O'Reilly Media, 2021. 412 p.
3. DevOps Knowledge Base. DevOps Info. URL: <https://devops.info/> (дата звернення 10.04.2025).
4. Docker Compose: Freehost Documentation URL: <https://freehost.com.ua/ukr/faq/articles/docker-compose/> (дата звернення 26.08.2025).
5. Docker Documentation. Docker, Inc. URL: <https://docs.docker.com/> (дата звернення 01.03.2025).
6. Flask Documentation. The Pallets Projects. URL: <https://flask.palletsprojects.com/en/stable/> (дата звернення 16.08.2025).
7. GitHub Actions Guide. Foxminded. URL: <https://foxminded.ua/github-actions/> (дата звернення 12.10.2025).
8. Goodfellow I., Bengio Y., Courville A. Deep Learning. MIT Press, 2016. 800 p.
9. Grafana Documentation. Dashboard JSON Model. URL: <https://grafana.com/docs/grafana/latest/visualizations/dashboards/build-dashboards/view-dashboard-json-model/> (дата звернення 12.09.2025).
10. Gunicorn Documentation. URL: <https://docs.gunicorn.org/en/stable/> (дата звернення 27.06.2025).
11. Hutter F., Kotthoff L., Vanschoren J. Automated Machine Learning: Methods, Systems, Challenges — Springer, 2019. — 219 p.
12. Merkel D. Docker: Lightweight Linux Containers for Consistent Development and Deployment // Linux Journal. — 2014. — No. 239. — P. 2–9.
13. NVIDIA Container Toolkit Documentation. URL: <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html> (дата звернення 04.08.2025).

14.PSUTIL Library Documentation. URL: <https://psutil.readthedocs.io/en/latest/> (дата звернення 25.10.2025).

15.Python Documentation, Version 3.10.19. URL: <https://docs.python.org/release/3.10.19/> (дата звернення 17.07.2025).

16.PyTorch Documentation. URL: <https://docs.pytorch.org/docs/stable/> (date of access: 21.08.2025).

17.PyTorch Vision Documentation. URL: <https://docs.pytorch.org/vision/stable/> (date of access: 02.09.2025).

18.Washizaki H., Fukazawa Y., Yamamoto T. MLOps: Overview, Definition, and Architecture. IEEE Access. — 2023. — Vol. 11. — P. 70651–70668.